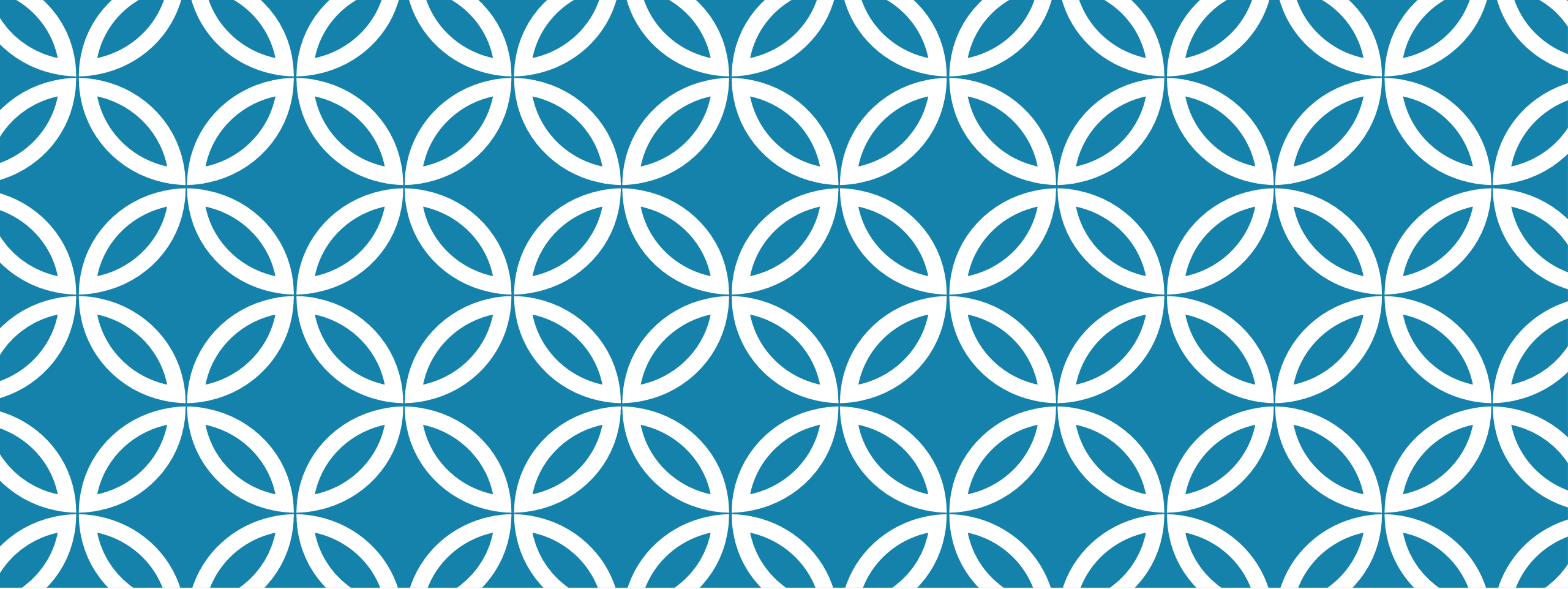




HADOOP ECOSYSTEM

Module 2

IT
ACCOMPLISHES
THE
FOLLOWING BIG
DATA TASKS:
1. MASSIVE DATA
STORAGE
2. FASTER
PROCESSING

The Apache™ Hadoop® project develops open-source software for reliable, scalable, distributed computing.

The Apache Hadoop software library is a framework that allows for the distributed processing of large data sets across clusters of computers using simple programming models. It is designed to scale up from single servers to thousands of machines, each offering local computation and storage. Rather than rely on hardware to deliver high-availability, the library itself is designed to detect and handle failures at the application layer, so delivering a highly-available service on top of a cluster of computers, each of which may be prone to failures.

Apache Hadoop

What is Apache Hadoop?

Apache Hadoop is an open-source software framework used for:

- Distributed computing
- Reliable storage
- Scalable processing of large data sets

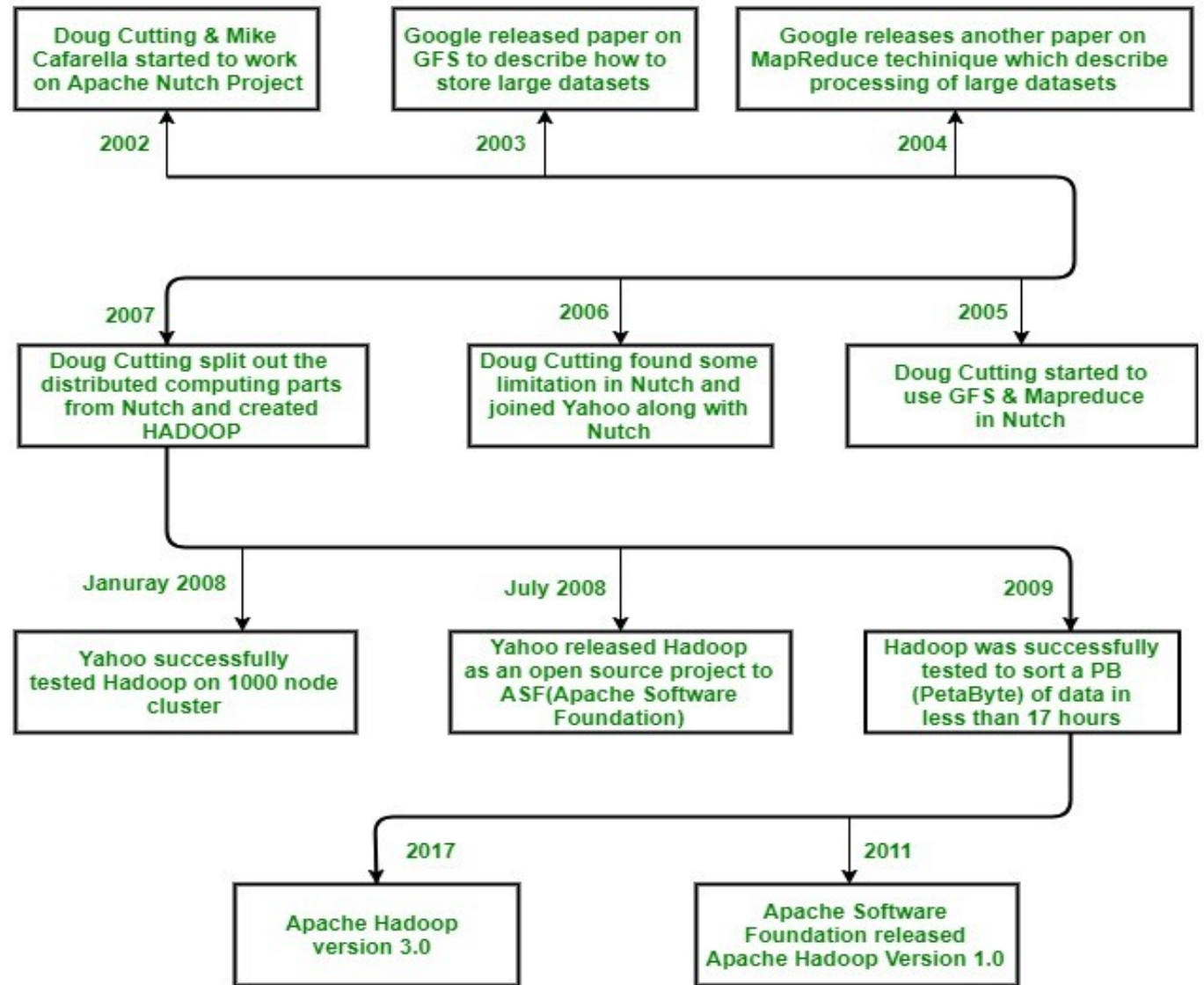
Key Features:

- Distributed Processing:
- Hadoop divides large data sets into smaller chunks and processes them across a cluster of computers, working in parallel.
- Simple Programming Model
- Developers can write programs using simple code, often based on the MapReduce model (which breaks tasks into "map" and "reduce" steps).

Why Hadoop is Important in Big Data Analytics:

- It allows companies to store and analyze petabytes of data efficiently.
- Supports real-time insights from massive data sets.
- Powers technologies behind major companies like Google, Yahoo, Facebook, etc. Scalability:
- It can scale from a single machine to thousands of machines, each with its own local computation and storage.
- Fault Tolerance: Instead of depending on expensive, fault-tolerant hardware:
- Hadoop is built to handle failures at the application level. If one machine fails, Hadoop redirects the task to another machine.

HISTORY OF HADOOP



Hadoop was started with **Doug Cutting and Mike Cafarella** in the year 2002 when they both started to work on Apache Nutch project. Apache Nutch project was the process of building a search engine system that can index 1 billion pages.

After a lot of research on Nutch, they concluded that such a system will cost around half a million dollars in hardware, and along with a monthly running cost of \$30, 000 approximately, which is very expensive. So, they realized that their project architecture will not be capable enough to the workaround with billions of pages on the web. So they were looking for a feasible solution which can reduce the implementation cost as well as the problem of storing and processing of large datasets.

In 2003, they came across a paper that described the architecture of Google's distributed file system, called **GFS (Google File System)** which was published by Google, for storing the large data sets.

Now they realize that this paper can solve their problem of storing very large files which were being generated because of web crawling and indexing processes. But this paper was just the half solution to their problem.

In 2004, Google published one more paper on the technique **MapReduce**, which was the solution of processing those large datasets.

Now this paper was another half solution for Doug Cutting and Mike Cafarella for their Nutch project. These both techniques (GFS & MapReduce) were just on white paper at Google. Google didn't implement these two techniques. Doug Cutting knew from his work on Apache Lucene (It is a free and open-source information retrieval software library, originally written in Java by Doug Cutting in 1999) that open-source is a great way to spread the technology to more people. So, together with Mike Cafarella, he started implementing Google's techniques (GFS & MapReduce) as open-source in the Apache Nutch project.

In 2005, Cutting found that Nutch is limited to only 20-to-40 node clusters. He soon realized two problems:
(a) Nutch wouldn't achieve its potential until it ran reliably on the larger clusters
(b) And that was looking impossible with just two people (Doug Cutting & Mike Cafarella).
The engineering task in Nutch project was much bigger than he realized. So he started to find a job with a company who is interested in investing in their efforts. And he found Yahoo!. Yahoo had a large team of engineers that was eager to work on this there project.

So in 2006, Doug Cutting joined Yahoo along with Nutch project. He wanted to provide the world with an open-source, reliable, scalable computing framework, with the help of Yahoo. So at Yahoo first, he separates the distributed computing parts from Nutch and **formed a new project Hadoop (He gave name Hadoop it was the name of a yellow toy elephant which was owned by the Doug Cutting's son. and it was easy to pronounce and was the unique word.)** Now he wanted to make Hadoop in such a way that it can work well on thousands of nodes. So with GFS and MapReduce, he started to work on Hadoop.

In 2007, Yahoo successfully tested Hadoop on a 1000 node cluster and start using it.

In January of 2008, Yahoo released Hadoop as an open source project to ASF(Apache Software Foundation). And in **July of 2008, Apache Software Foundation successfully tested a 4000 node cluster with Hadoop.**

In 2009, Hadoop was successfully tested to sort a PB (PetaByte) of data in less than 17 hours for handling billions of searches and indexing millions of web pages. And **Doug Cutting left the Yahoo and joined Cloudera to fulfill the challenge of spreading Hadoop to other industries.**

Hadoop's history shows us:

- How **open-source collaboration** can solve big problems.
- How **research (like Google's GFS and MapReduce)** can inspire a powerful platform.
- How companies (like **Yahoo! and Apache Software Foundation**) can collectively bring a project to maturity.

Today, **Hadoop** forms a key base for many **Big Data and Data Analytics platforms** used by companies all over the world.

WHY HADOOP

1. Flexible:As it is a known fact that only 20% of data in organizations is structured, and the rest is all unstructured, it is very crucial to manage unstructured data which goes unattended.

Hadoop manages different types of Big Data, whether structured or unstructured, encoded or formatted, or any other type of data and makes it useful for decision making process.

2. Scalable:Hadoop is a scalable platform, in the sense that new nodes can be easily added in the system as and when required without altering the data formats, how data is loaded, how programs are written, or even without modifying the existing applications.

Hadoop is an open source platform and runs on industry-standard hardware. Moreover, Hadoop is also fault tolerant – this means, even if a node gets lost or goes out of service, the system automatically reallocates work to another location of the data and continues processing as if nothing had happened!

3. Building more efficient data economy:Hadoop has revolutionized the processing and analysis of big data world across. Till now, organizations were worrying about how to manage the non-stop data overflowing in their systems.



4. Robust Ecosystem: Hadoop has a very robust and a rich ecosystem that is well suited to meet the analytical needs of developers, web start-ups and other organizations.

The Ecosystem consists of various related projects such as MapReduce, Hive, HBase, Zookeeper, HCatalog, Apache Pig, which make it very competent to deliver a broad spectrum of services

5. Hadoop is getting more “Real-Time”! :

Did you ever wonder how to stream information into a cluster and analyze it in real time?

Hadoop has the answer for it. Yes, the competencies are getting more and more real-time. It also provides a standard approach to a wide set of APIs for big data analytics comprising MapReduce, query languages and database access, and so on.

CORE COMPONENTS OF HADOOP

The 3 core components of the Apache Software Foundation's Hadoop framework are:

1. MapReduce – A software programming model for processing large sets of data in parallel
2. HDFS – The Java-based distributed file system that can store all kinds of data without prior organization.
3. YARN – A resource management framework for scheduling and handling resource requests from distributed applications.

HADOOP ECOSYSTEM

- These ecosystem components are different services deployed by the various enterprise.
- We can integrate these to work with a variety of data.
- Each of the Hadoop Ecosystem Components is developed to deliver explicit function.
- And each has its own developer community and individual release cycle.



Hadoop Ecosystem



Table & schema
Management



Pig
(Scripting)



Hive
(Sql Query)



(Machine
Learning)



Drill
(Interactive
Analysis)



AVRO
(JSON)



Thrift
(Cross
Language
Service)

A P A C H E
HBASE

HBASE
(Columnar
Store)



Sqoop
(Data Collection)

oozie

oozie

(Work flow)



Zookeeper
(Coordination)



Ambari

Apache Ambari
(Management
& Monitoring)

Mapreduce
(Data Processing)



Yarn
(Cluster Resource Management)

HDFS
(Hadoop Distributed File system)



Flume
(Data Collection)

HDFS (Hadoop Distributed File System):

- **What it does:** Stores large amounts of data across many machines in a distributed manner.
- **How:** It splits files into blocks and saves them across multiple nodes. This guarantees **fault-tolerance** (so if a few machines break down, your data is still safe).
- **Importance:** Reliable, fault-tolerant, and designed for big data.
- **Example:**
Picture you have 100 GB of social media messages. Instead of putting them all on a single hard drive, you cut them into 128 MB blocks and store them across 100 machines in your data center.

YARN (Yet Another Resource Negotiator):

- **What it does:** Manages resources (like CPU, RAM) across the Hadoop clusters.
- **How:** It schedules and allocates resources to different applications efficiently.
- **Importance:** Ensuring that all computing resources are used optimally.
- **Example:**
If two big data programs are competing for resources, YARN guarantees that each gets its fair share — much like a hotel receptionist allocating rooms.

MapReduce:

- **What it does:** Processes large amounts of data in parallel.
- **How:** Split, process, combine — following two phases (Map and Reduce).
- **Importance:** Allows you to perform heavy computations efficiently.
- **Example:**
If you want to count word frequencies in 1000 documents, you can **map** (each document parses its own words) and then **reduce** (combining total count for each word).

Table & Schema Management:

- **What it does:** Stores and maintains structure (table schema) for your data.
- **Importance:** Helps you keep your data well organized for easy querying.
- **Example:**
Picture you have a table with customer IDs, names, and emails — this metadata lets you quickly identify what each column means

Pig (Scripting Platform):

- **What it does:** Allows you to process large data sets with a scripting language called Pig Latin.
- **Importance:** Easier to use than low-level code — saves developer's time.
- **Example:**
Transform raw server logs into cleaned data sets with just a few lines of Pig script instead of heavy Java code.

Hive (SQL Query):

- **What it does:** Allows you to perform SQL-like queries directly on big data in Hadoop.
- **Importance:** Helps people with SQL skills work efficiently with big data.
- **Example:**
Analyze sales data to compute total sales per region with a simple `SELECT SUM(sales) GROUP BY region`.

Drill (Interactive Analysis):

- **What it does:** Allows you to perform self-service and interactive data analysis without needing extensive programming skills.
- **Importance:** Helps business users quickly extract information from big data.
- **Example:**
Analyze customer reviews to identify patterns in real time (like most frequently used words related to satisfaction).

Avro (JSON):

- **What it does:** Stores data in a compact format (JSON with schema).
- **Importance:** Allows for schema evolution and data interoperability.
- **Example:**
Consider adding a new field, “age”, to your customer messages — with Avro, you can do this without affecting your existing data format.

Thrift (Cross-Language Service):

- **What it does:** Facilitates communication between different programming languages. Used to define data types and service interfaces in a language-neutral way
- **Importance:** Allows services implemented in different languages (like Java, C++, or Python) to work together.
- **Example:**
Your Java application can connect to a Thrift service implemented in C++ without needing to translate code manually

Step 1: Write a .thrift file

```
// customer_service.thrift service CustomerService { string getCustomerName(1: i32 customerId) }
```

Step 2: Use Thrift to generate code:

```
thrift --gen java customer_service.thrift  
thrift --gen cpp customer_service.thrift
```

Zookeeper (Coordinator):

- **What it does:** Helps keep track of network configuration, synchronization, and coordination between services.
- **Importance:** Prevents conflicting operations — for instance, two services trying to become the “master” at the same time.
- **Example:**
Managing a group of messaging brokers, making sure only **one** is acting as master while the rest follow.

Oozie (Workflow):

- **What it does:** Allows you to schedule and manage complex workflows composed of multiple big data jobs.
- **Importance:** Helps you automate routines and manage dependencies between jobs.
- **Example:**
An ETL pipeline: first parses raw data with Pig, then performs computations with Hive, then stores the results in HBase.

Flume (Data Collection):

- **What it does:** Facilitates **ingestion of large amounts of event data and logs into HDFS.**
- **Importance:** Reliable delivery and transformation of large volumes of data in real time.
- **Example:**
Collecting server logs from many sources (web servers, routers) and sending them into Hadoop for further processing.

Sqoop (Data Collection):

- **What it does:** Transfers **structured data** from relational databases (like MySQL or Oracle) into Hadoop (and back).
- **Importance:** Allows you to combine enterprise databases with big data for a complete view.
- **Example:**
Importing customer data from a production database into HDFS for extensive analysis alongside other big data sources

HBase (Columnar Store):

- **What it does:** Stores large amounts of data in columns instead of rows — a **NoSQL database designed for fast, random read and write operations.**
- **Importance:** Allows real-time workloads alongside batch processing.
- **Example:** Managing messages or sensor data for real-time applications — for instance, a messaging platform with high-throughput delivery.

Hadoop Ecosystem

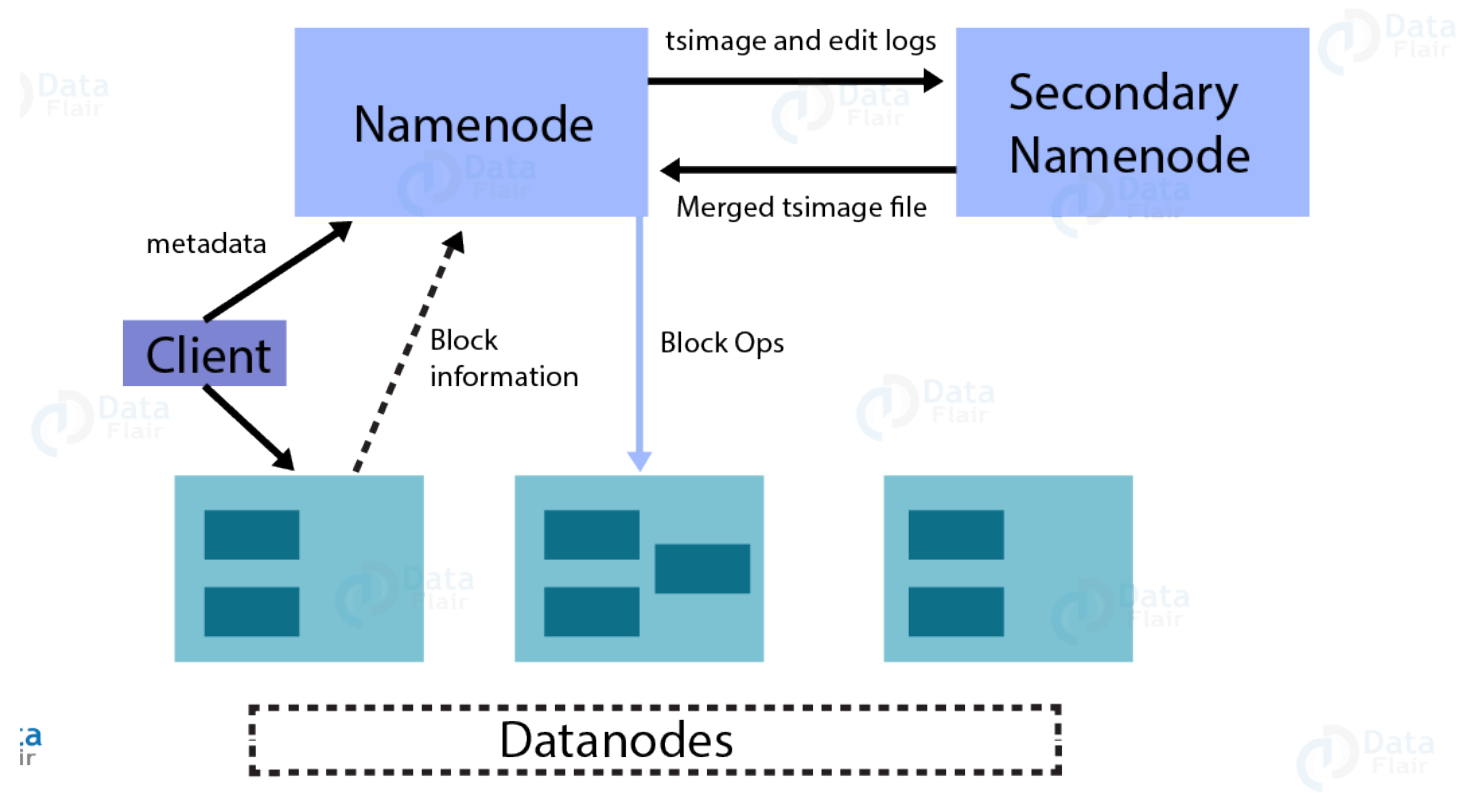
- Built on top of Hadoop core to provide additional functionalities:
 - **Apache Pig:** Scripting platform for analyzing large datasets.
 - **Apache Hive:** Data warehouse infrastructure built on Hadoop.
 - **Apache HBase:** Non-relational, distributed database that runs on HDFS.
 - **Apache Spark:** In-memory data processing engine for speed and analytics.
 - **Apache Sqoop:** Tool for transferring data between Hadoop and relational databases.
 - **Flume:** Collects and transfers large volumes of log data into Hadoop.
 - **Zookeeper:** Manages and coordinates distributed applications.

Summary:

The **Hadoop ecosystem** comprises a set of components and tools, each designed to solve a particular big data challenge — storage, processing, coordination, data transfer, or real-time access.

All these components **work together**, forming a powerful platform for big data applications.

HADOOP DISTRIBUTED FILE SYSTEM



HDFS ARCHITECTURE

Apache HDFS or Hadoop Distributed File System is a block-structured file system where each file is divided into blocks of a pre-determined size.

These blocks are stored across a cluster of one or several machines.

Apache Hadoop HDFS Architecture follows a Master/Slave Architecture, where a cluster comprises of a single NameNode (Master node) and all the other nodes are DataNodes (Slave nodes).

HDFS can be deployed on a broad spectrum of machines that support Java.

Though one can run several DataNodes on a single machine, but in the practical world, these DataNodes are spread across various machines.

HDFS ARCHITECTURE

- **NameNode** = Directory/Index manager
- **DataNodes** = Actual file storage
- **Client** = User requesting or writing data
- **Replication** = Backup copies to ensure safety
- **Rack Awareness** = Distributes copies across different groups of machines (racks)
- to avoid loss during hardware failures.

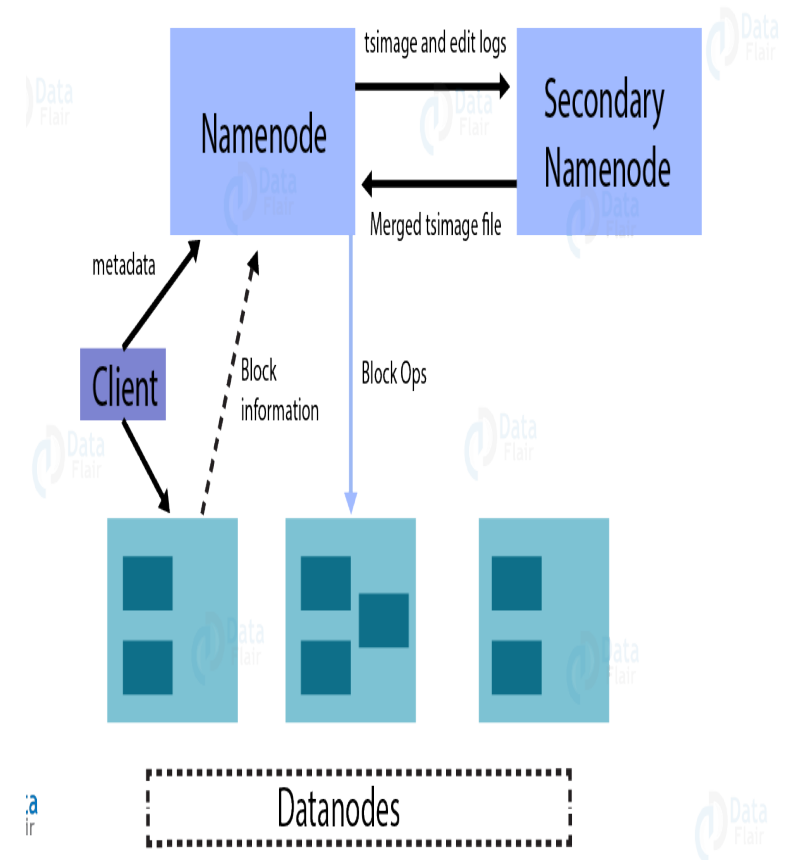
HADOOP DISTRIBUTED FILE SYSTEM

1. Client

- The **client** is the user or program that wants to **store or read data** in HDFS.
- It sends requests to the **NameNode**.

2. NameNode

- The **NameNode** is the **master** of HDFS.
- It **does not store actual data**, but **stores metadata**.
- It keeps **metadata** — information like:
 - File names
 - Locations
 - How many copies (replicas) of each file exist



Name Node

NameNode is the master node in the Apache Hadoop HDFS Architecture that maintains and manages the blocks present on the DataNodes (slave nodes).

NameNode is a very highly available server that manages the File System Namespace and controls access to files by clients.

The HDFS architecture is built in such a way that the user data never resides on the NameNode. The data resides on DataNodes only.

Functions of NameNode:

- It is the master daemon that maintains and manages the DataNodes (slave nodes)
- It records the metadata of all the files stored in the cluster, e.g. The location of blocks stored, the size of the files, permissions, hierarchy, etc. There are two files associated with the metadata:

Name Node

- It records each change that takes place to the file system metadata. For example, if a file is deleted in HDFS, the NameNode will immediately record this in the EditLog.
- It regularly receives a Heartbeat and a block report from all the DataNodes in the cluster to ensure that the DataNodes are live.
- It keeps a record of all the blocks in HDFS and in which nodes these blocks are located.
- The NameNode is also responsible to take care of the **replication factor** of all the blocks.
- In **case of the DataNode failure**, the NameNode chooses new DataNodes for new replicas, balance disk usage and manages the communication traffic to the DataNodes.

Data Node

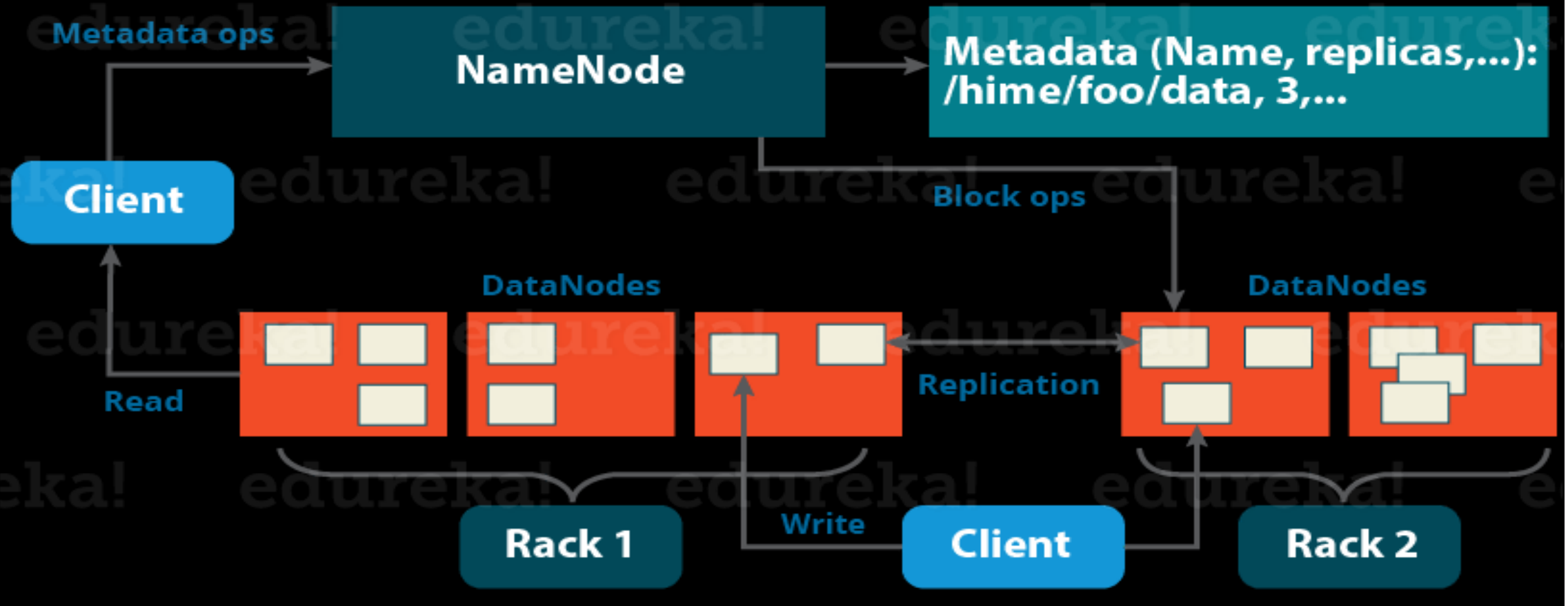
DataNode:

- DataNodes are the slave nodes in HDFS. Unlike NameNode, DataNode is a commodity hardware, that is, a non-expensive system which is not of high quality or high-availability.
- The DataNode is a block server that stores the data in the local file ext3 or ext4.
- These are the actual data storage machines.
- Each DataNode stores blocks of the files.
- They regularly send heartbeat signals to the NameNode to say they are alive.
- These are the actual storage nodes (where the real data blocks are stored).
- They are divided into Racks (Rack 1 and Rack 2 in the diagram).

Functions of DataNode:

- These are slave daemons or process which runs on each slave machine.
- The actual data is stored on DataNodes.
- The DataNodes perform the low-level read and write requests from the file system's clients.
- They send heartbeats to the NameNode periodically to report the overall health of HDFS, by default, this frequency is set to 3 seconds.

HDFS Architecture



What Happens When a Client Wants to Write Data?

1. The **Client** talks to the **NameNode** and asks where to write.
2. The **NameNode** replies with info like:
"Store this block in DataNode A, and replicate it in DataNode B and C."
3. The **Client** writes data blocks to **DataNodes** in **Rack 1**.
4. **Data is replicated** (copied) to **other DataNodes** in **Rack 2** for fault tolerance.

What Happens When a Client Wants to Read Data?

1. The Client first asks the NameNode:

- "Where is my file /home/foo/data?"

2. The NameNode returns block locations.

The Client reads data directly from the DataNodes (fast and parallel access).

Types of Operations:

- **Metadata ops:** Communication between Client and NameNode (no actual data, just info).
- **Block ops:** Data movement between NameNode and DataNodes (for managing data).
- **Replication:** HDFS automatically makes 3 copies of each block and stores them on different nodes and racks to prevent data loss.

Secondary NameNode

- **Misleading name** — it's **not a backup** of the NameNode.
- Its real job is to help the NameNode by:
 - Periodically **combining metadata (FsImage) and logs (Edits)** to reduce the load on the NameNode.
 - FsImage: It contains the complete state of the file system namespace since the start of the NameNode.
 - EditLogs: It contains all the recent modifications made to the file system with respect to the most recent FsImage.

The **Secondary NameNode** is a **checkpointing helper** for the NameNode.

1Downloads FsImage and EditLogs from the NameNode

2Merges them into a new FsImage (updated snapshot)

3Uploads the new FsImage back to the NameNode

- This helps keep metadata **compact and efficient**, ensuring **faster recovery** and **better performance**.

<https://youtu.be/LVYCjmVzT3A?feature=shared>

SECONDARY NAMENODE:

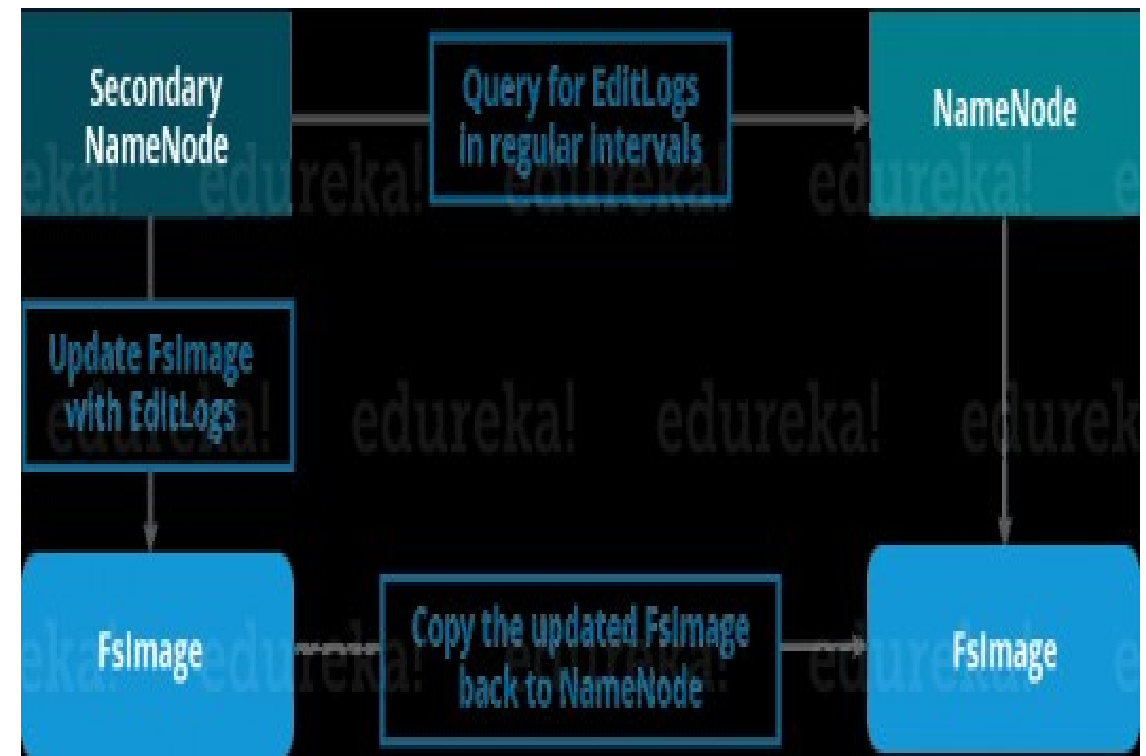
The Secondary NameNode works concurrently with the primary NameNode as a helper daemon.

- The Secondary NameNode is one which constantly reads all the file systems and metadata from the RAM of the NameNode and writes it into the hard disk or the file system.

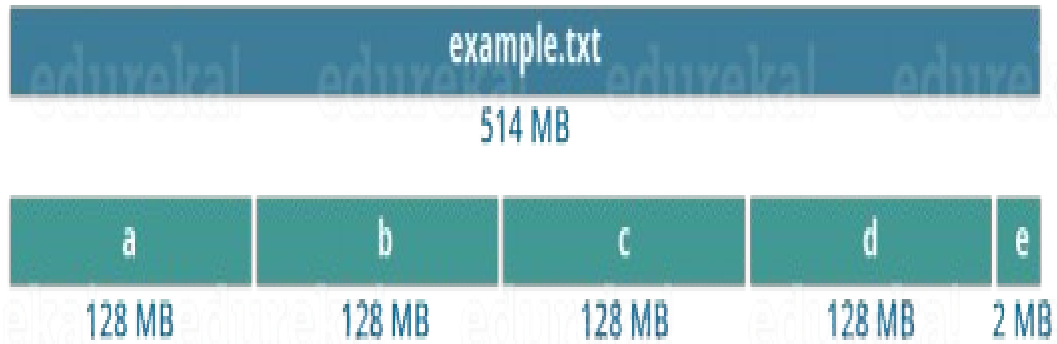
- It is responsible for combining the EditLogs with FsImage from the NameNode.

- It downloads the EditLogs from the NameNode at regular intervals and applies to FsImage. The new FsImage is copied back to the NameNode, which is used whenever the NameNode is started the next time.

Hence, Secondary NameNode performs regular checkpoints in HDFS. Therefore, it is also called **CheckpointNode**.



BLOCKS:



Blocks are the nothing but the smallest continuous location on your hard drive where data is stored. In general, in any of the File System, you store the data as a collection of blocks.

Similarly, HDFS stores each file as blocks which are scattered throughout the Apache Hadoop cluster. The default size of each block is 128 MB

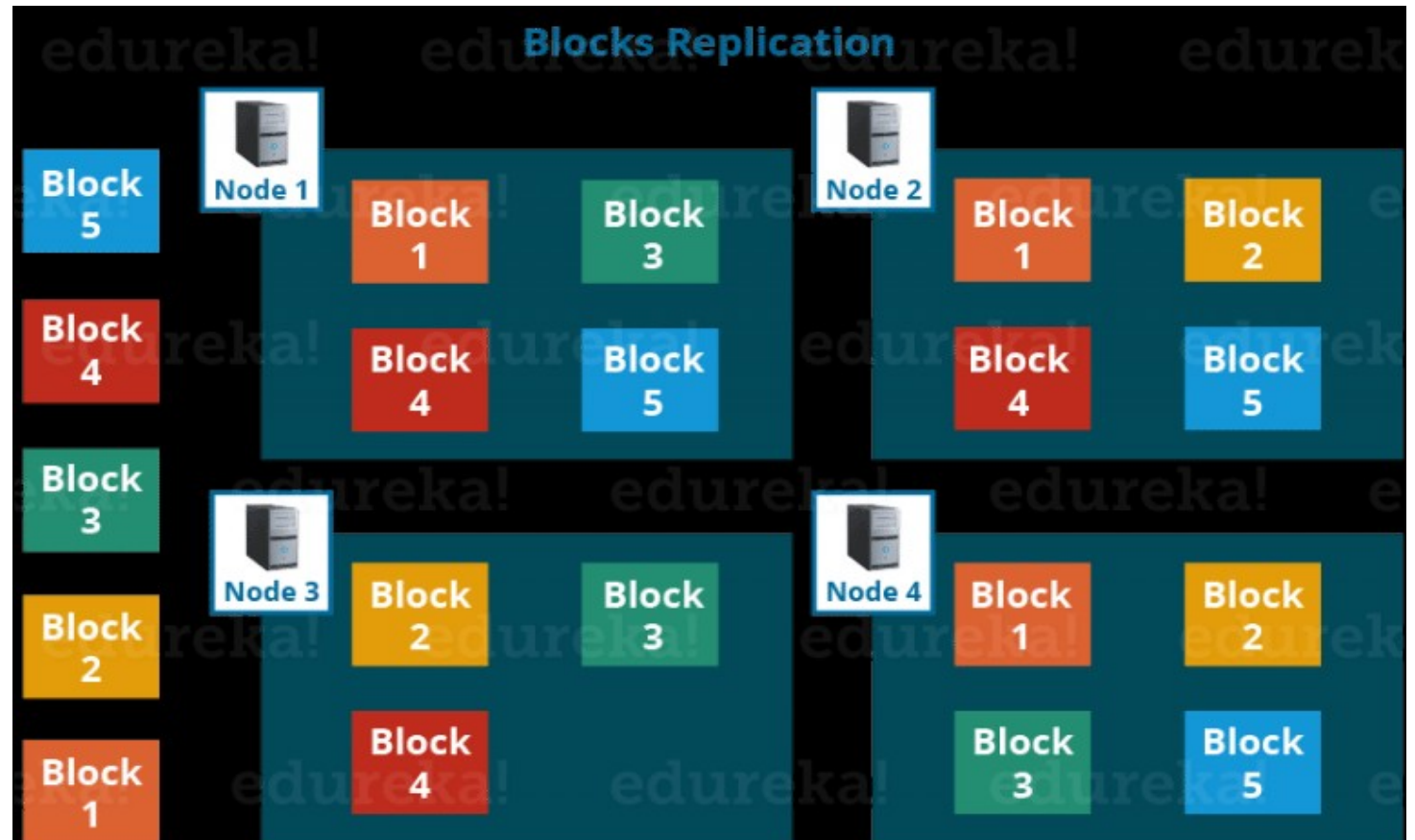
REPLICATION MANAGEMENT:

HDFS provides a reliable way to store huge data in a distributed environment as data blocks.

The blocks are also replicated to provide fault tolerance.

The default replication factor is 3 which is again configurable.

So, as you can see in the figure below where each block is replicated three times and stored on different DataNodes (considering the default replication factor)



RACK AWARENESS:

NameNode also ensures that all the replicas are not stored on the same rack or a single rack.

It follows an in-built Rack Awareness Algorithm to reduce latency as well as provide fault tolerance.

Considering the replication factor is 3, the Rack Awareness Algorithm says that the first replica of a block will be stored on a local rack and the next two replicas will be stored on a different (remote) rack but, on a different DataNode within that (remote) rack

Rack Awareness Algorithm

Block A :  Block B:  Block C: 



Client Node (Client JVM) Side:

1. Create Request Initiated by HDFS Client:

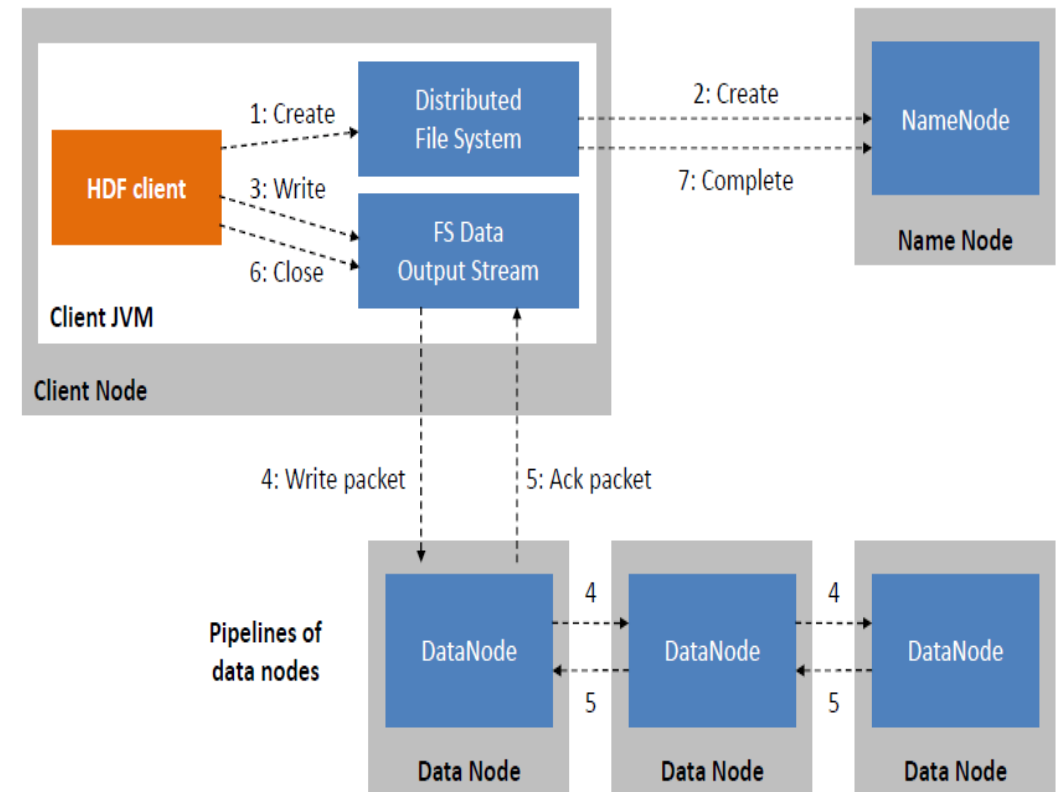
- The HDFS client calls the create() method on the Distributed File System.
- It sends a request to the NameNode to create a new file in HDFS.

2. Create Request Sent to NameNode:

- NameNode checks if the file already exists and if the user has the necessary permissions.
- If valid, it creates metadata and returns details about which DataNodes to write to.

3. Write Operation Begins:

- Client starts writing data via FSDataOutputStream to the first DataNode.
- Data is broken into packets and sent down a pipeline of DataNodes.



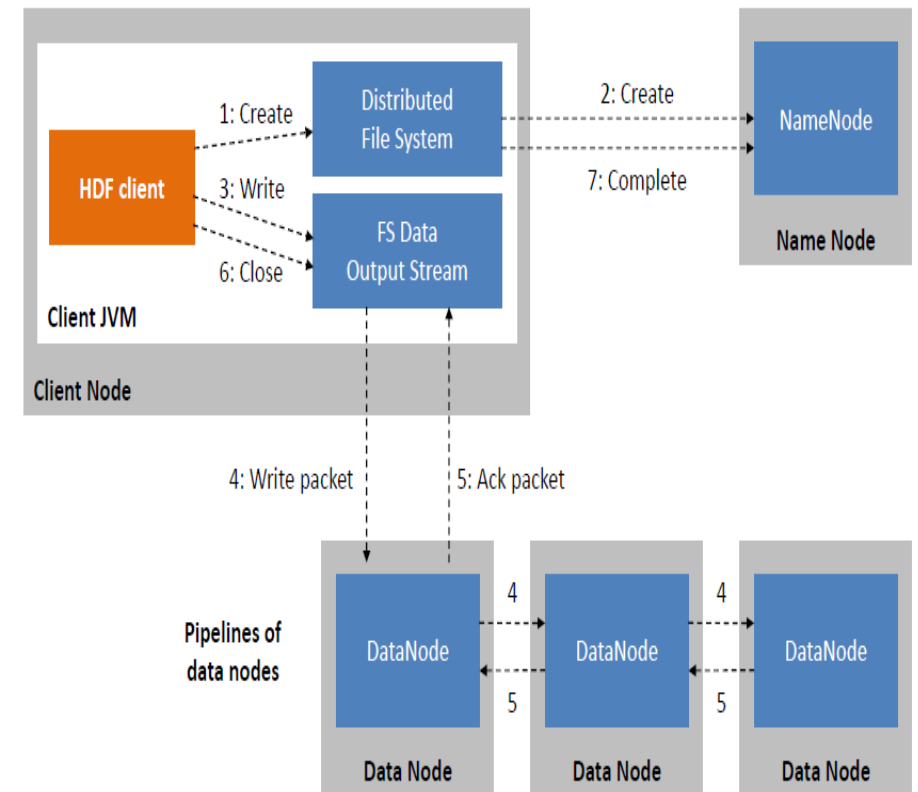
DataNode Pipeline (Write and Acknowledgment):

4. Write Packet:

- The client sends a block (packet) of data to the first DataNode.
- The first DataNode forwards the packet to the second, and so on (chained transfer).

5. Acknowledge Packet (ACK):

- Once the last DataNode in the pipeline receives the packet, it sends an ACK back up the pipeline.
- This confirmation moves back from the last DataNode to the client.



HDFS Write Process with Acknowledgement (Simplified)

1. Client Sends Block (BLK A)

The HDFS Client wants to write a file (or block "A") to HDFS.

2. Block is Written to the First DataNode (DataNode 1)

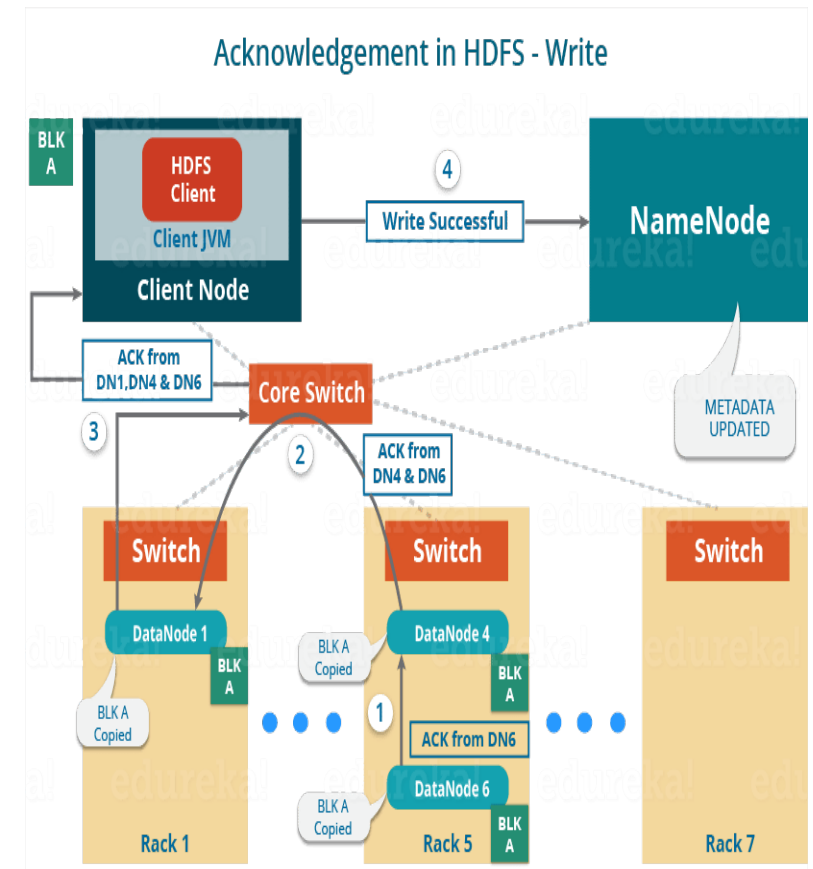
- The client writes the block to **DataNode 1** (in **Rack 1**).
- DataNode 1 stores it and then forwards it to the next DataNode in the replication chain.

3. Block is Forwarded to DataNode 4

- DataNode 1 sends block A to **DataNode 4** (in **Rack 5**).
- After storing it, DataNode 4 forwards it to another DataNode.

4. Block is Forwarded to DataNode 6

- DataNode 4 sends block A to **DataNode 6** (also in **Rack 5**).
- Now, all 3 replicas of block A are stored: one each on DataNodes 1, 4, and 6.



5. Acknowledgement from DataNode 6 to DataNode 4

Once DataNode 6 receives and stores the block, it sends an ACK (acknowledgement) back to DataNode 4.

6. Acknowledgement from DataNode 4 to DataNode 1

DataNode 4 receives the ACK from DataNode 6 and sends its own ACK (and DataNode 6's) to DataNode 1.

7. DataNode 1 Sends Final ACK to Client

After receiving all ACKs, DataNode 1 sends the final ACK to the client.

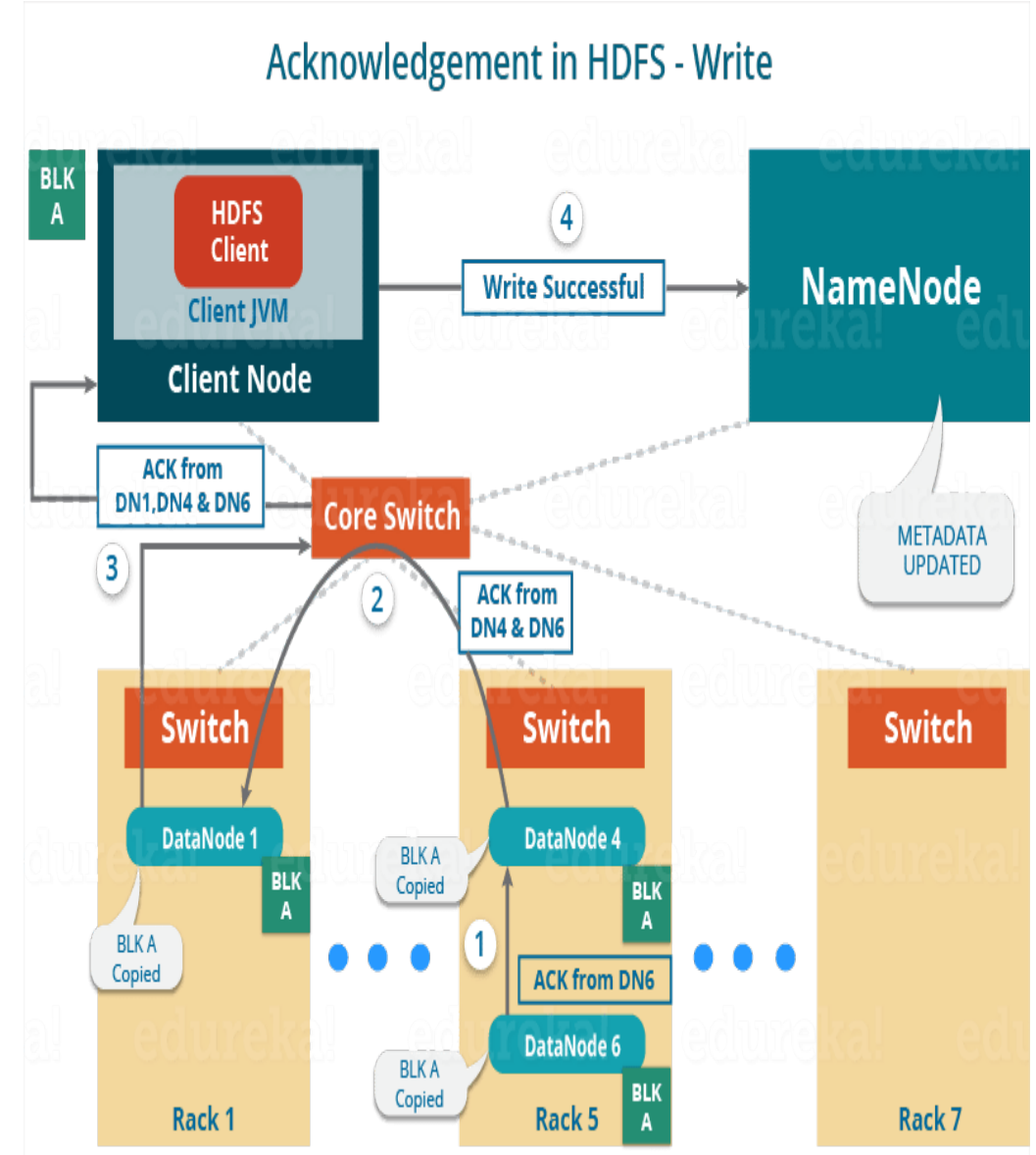
8. Client Notifies NameNode

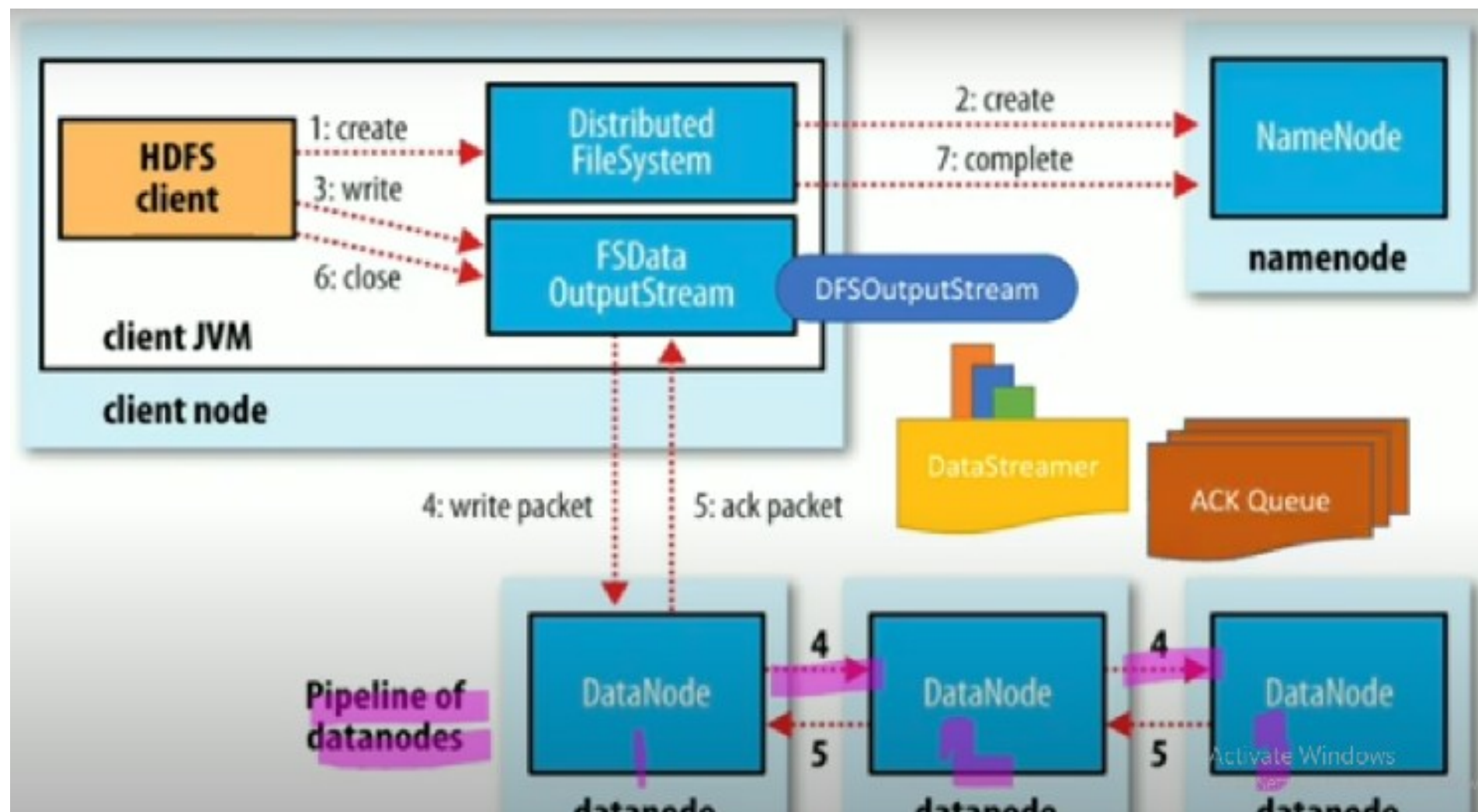
The client informs the **NameNode** that the write is successful.

9. NameNode Updates Metadata

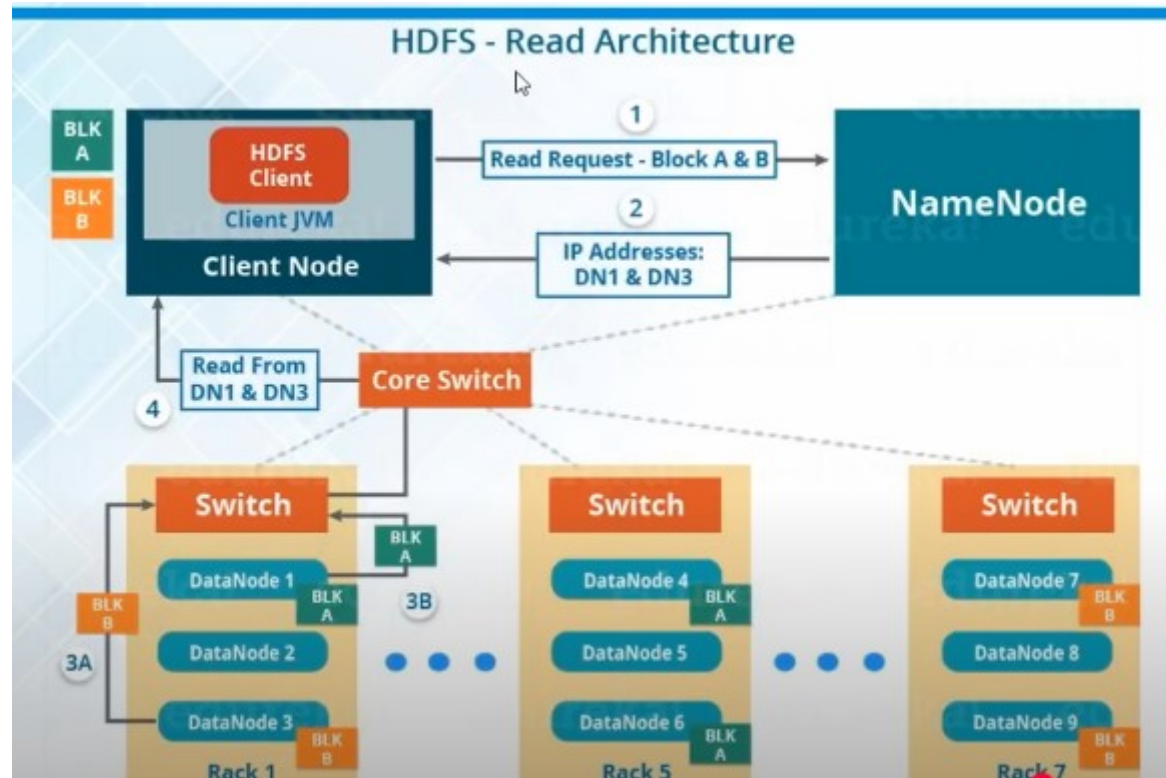
NameNode updates its metadata (file location, block information, etc.).

10. Write Operation Completed Successfully!





Read Architecture



HDFS Read

HDFS Read Operation (Step-by-Step)

1. Open File Request

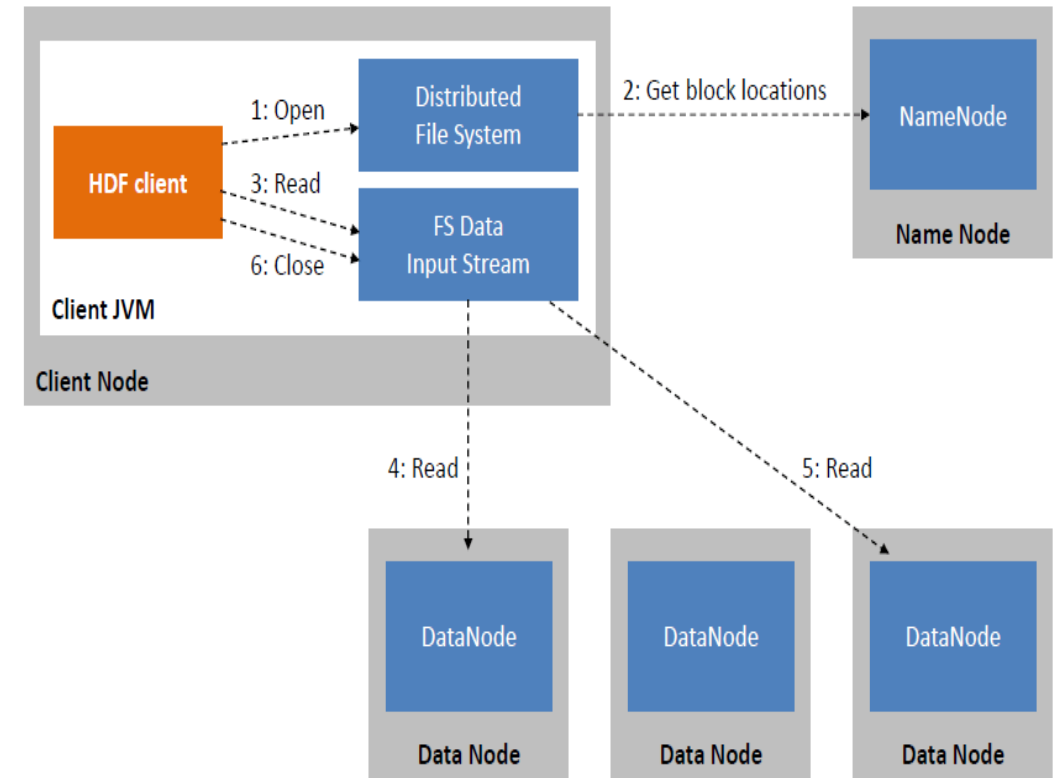
1. The **HDFS client** requests to open a file from the **Distributed File System**.

2. Get Block Locations

1. The Distributed File System contacts the **NameNode** to get the **block locations** (i.e., where parts of the file are stored in different DataNodes).

3. Start Reading

1. Once block locations are known, the client starts reading the file via the **FS Data Input Stream**.



Read from DataNode

The client reads the required block(s) directly from the **nearest DataNode** (based on network proximity).

Continue Reading

- If the next block is stored on another DataNode, the client reads from that **next DataNode**.

Close File

- Once reading is done, the client **closes** the file.

Summary:

- NameNode only gives **metadata** (not the file content).
- Actual **data is read directly from DataNodes**.
- HDFS ensures **fault tolerance** by storing multiple block replicas.

Overview of Apache Tools

1. Apache Spark

1. Fast, in-memory distributed data processing framework.
2. Supports Java, Scala, Python, and R.
3. Offers modules for SQL, streaming, ML, and graph processing.

2. Apache Pig

1. High-level scripting language (Pig Latin) for querying and analyzing large datasets.
2. Converts scripts into MapReduce jobs.

3. Apache Hive

1. SQL-like interface for querying data stored in HDFS.
2. Suitable for data summarization and ad hoc querying.

4. Apache HBase

1. NoSQL database modeled after Google's Bigtable.
2. Provides real-time read/write access to large datasets.

5. Apache Sqoop

1. Command-line interface tool to transfer data between Hadoop and RDBMS.
2. Supports MySQL, PostgreSQL, Oracle, SQL Server, etc.

Example: E-commerce Website Log Analysis

We want to analyze customer behavior from **website logs** and **transaction data** to improve your online store.

Step-by-step Workflow with Hadoop Ecosystem Tools:

1. Flume – Collect Website Logs

- **What it does:** Automatically collects huge amounts of **log data** (like user clicks, page views) from web servers.
- **Example:** When a user visits or buys something, it creates log entries.
- **How it works:** Flume sends this data into **HDFS** (Hadoop Distributed File System).

2. Sqoop – Import Transaction Data

- **What it does:** Imports data from **MySQL**(name, email, etc.). (relational database with order history) into **HDFS**.
- **How it works:** Runs SQL queries on MySQL and stores the result into Hadoop.

Contd..

3. HDFS – Central Data Storage

- Now both log data (from Flume) and transaction data (from Sqoop) are **stored in HDFS**. (the heart of the Hadoop ecosystem).

4. Hive – Query Data Like SQL

Hive runs SQL queries on data stored in HDFS.

- **What it does:** Lets analysts write SQL-like queries to analyze the combined data in HDFS.
- **Example:** “Find top 10 products viewed but not purchased.”

Contd..

5. Pig – Process Complex Data

- **What it does:** Developers can write Pig scripts (instead of Java) to perform **data transformations** (e.g., filter, join, group).
- **Example:** Filter only the users who abandoned carts.

6. Spark – Fast Data Processing

- **What it does:** Processes data **faster than MapReduce** by using **in-memory computing**.
- **Use case:** You want to build a **real-time dashboard** of active users.

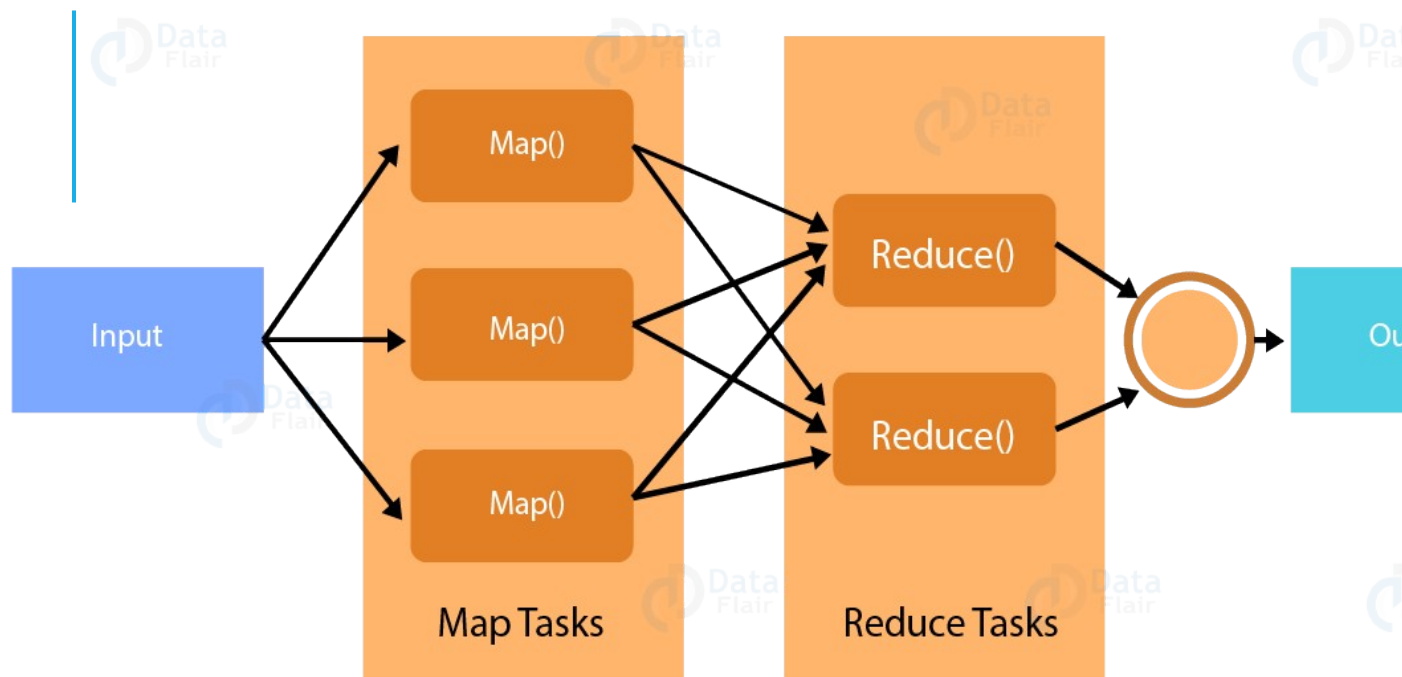
Contd..

7. HBase – Store Processed Data for Quick Access

- **What it does:** Stores frequently accessed data (like user profiles) in a **NoSQL** format for **fast read/write**.
- **Why:** Faster than HDFS for individual lookups.

8. Zookeeper – Coordinator

- **What it does:** Helps manage and coordinate all these distributed components (like making sure all nodes are synced and up).
- **Use case:** Ensures consistency when multiple users/processes access Hive or HBase.



MAPREDUCE

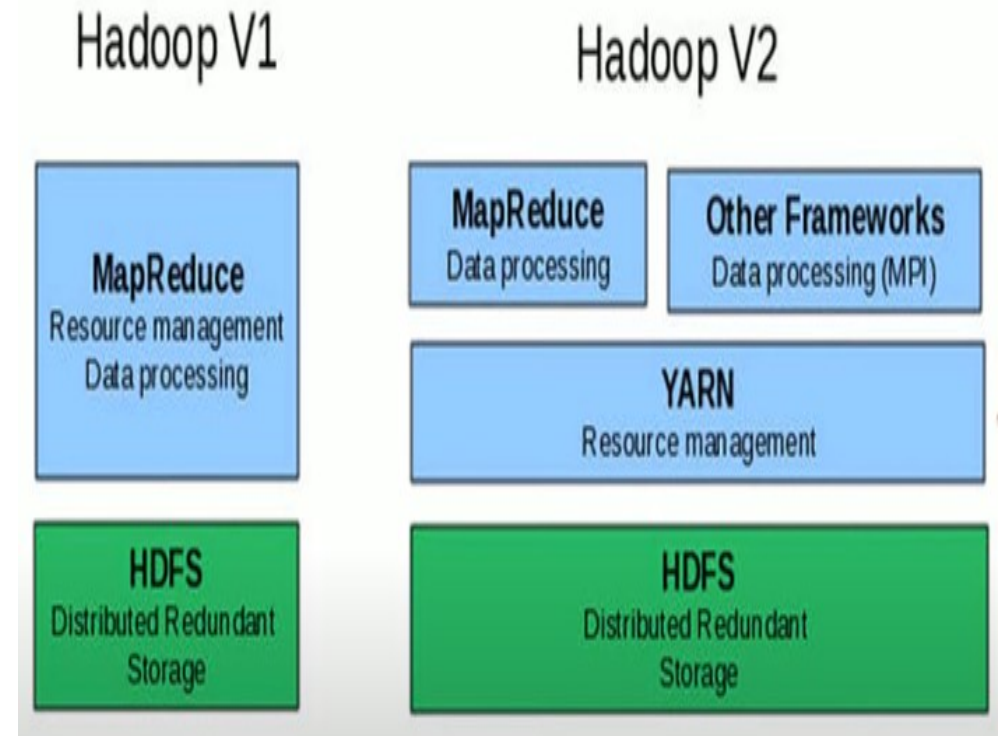
Map Phase – This phase takes input as key-value pairs and produces output as key-value pairs. It can write custom business logic in this phase. Map phase processes the data and gives it to the next phase.

Reduce Phase – The MapReduce framework sorts the key-value pair before giving the data to this phase. This phase applies the summary type of calculations to the key-value pairs.

MapReduce is the data processing component of Hadoop. It applies the computation on sets of data in parallel thereby improving the performance. MapReduce works in two phases –

YARN

- Yarn which is short for Yet Another Resource Manager.
- It is like the operating system of Hadoop as it monitors and manages the resources. Yarn came into the picture with the launch of Hadoop 2.x in order to allow different workloads.
- It handles the workloads like stream processing, interactive processing, batch processing over a single platform.
- Yarn has two main components – Node Manager and Resource Manager.



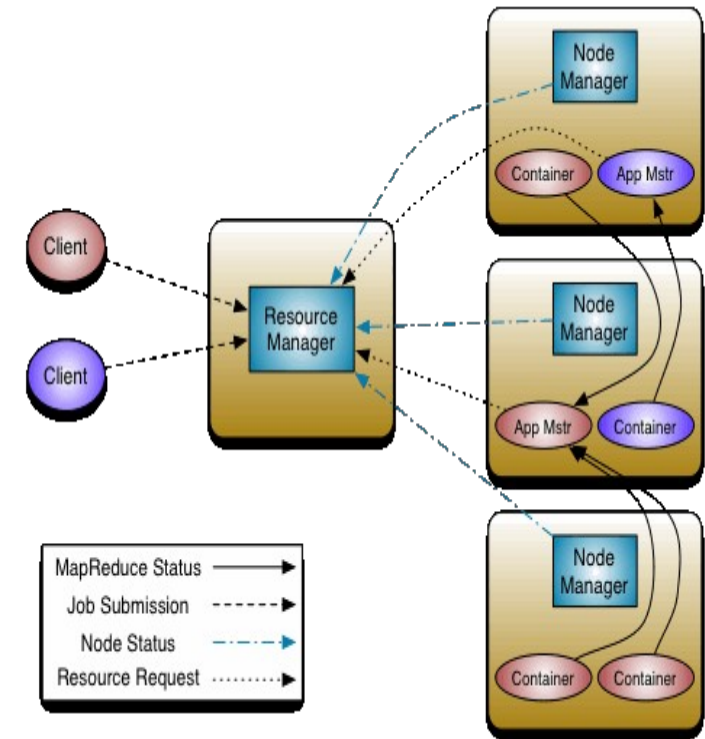
YARN

1. Client

- The **client** submits a job (like a MapReduce job) to **Resource Manager**.

2. Resource Manager (RM)

- The **brain** of YARN.
- It **receives jobs** and decides **which node** (computer) should run it.
- It doesn't run tasks itself — it just **allocates resources** (CPU, memory).



YARN

3. Node Manager (NM)

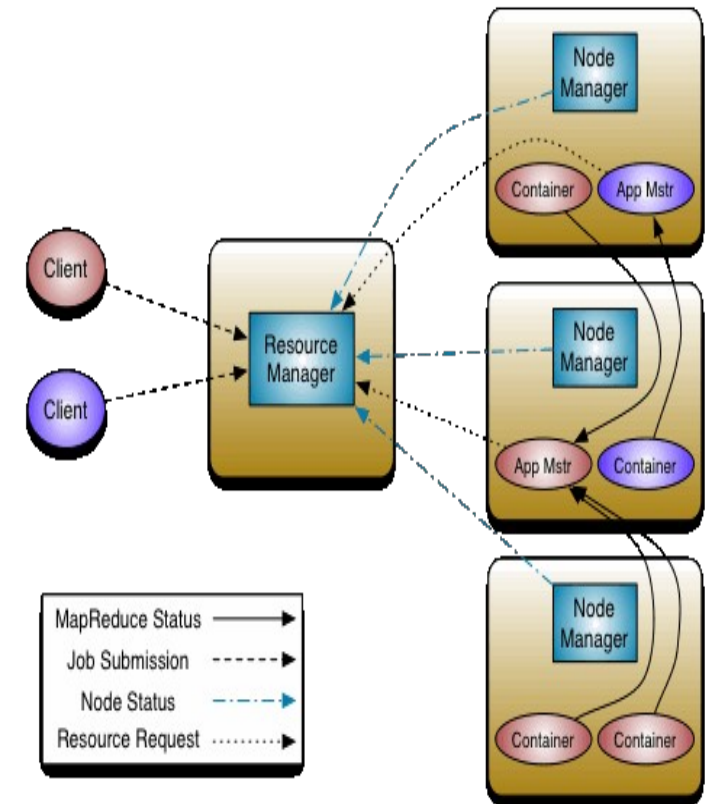
- Each machine in the cluster has a **Node Manager**.
- It **monitors resources** (CPU, memory) on its machine and **runs tasks in containers**.

4. Containers

- A **container** is like a small box where a part of the application runs.
- YARN puts jobs into these containers to isolate and manage resources.

5. Application Master (App Mstr)

- For each job, YARN creates an **Application Master** inside a container.
- It **manages the job** — keeps track of tasks, checks status, handles failures.

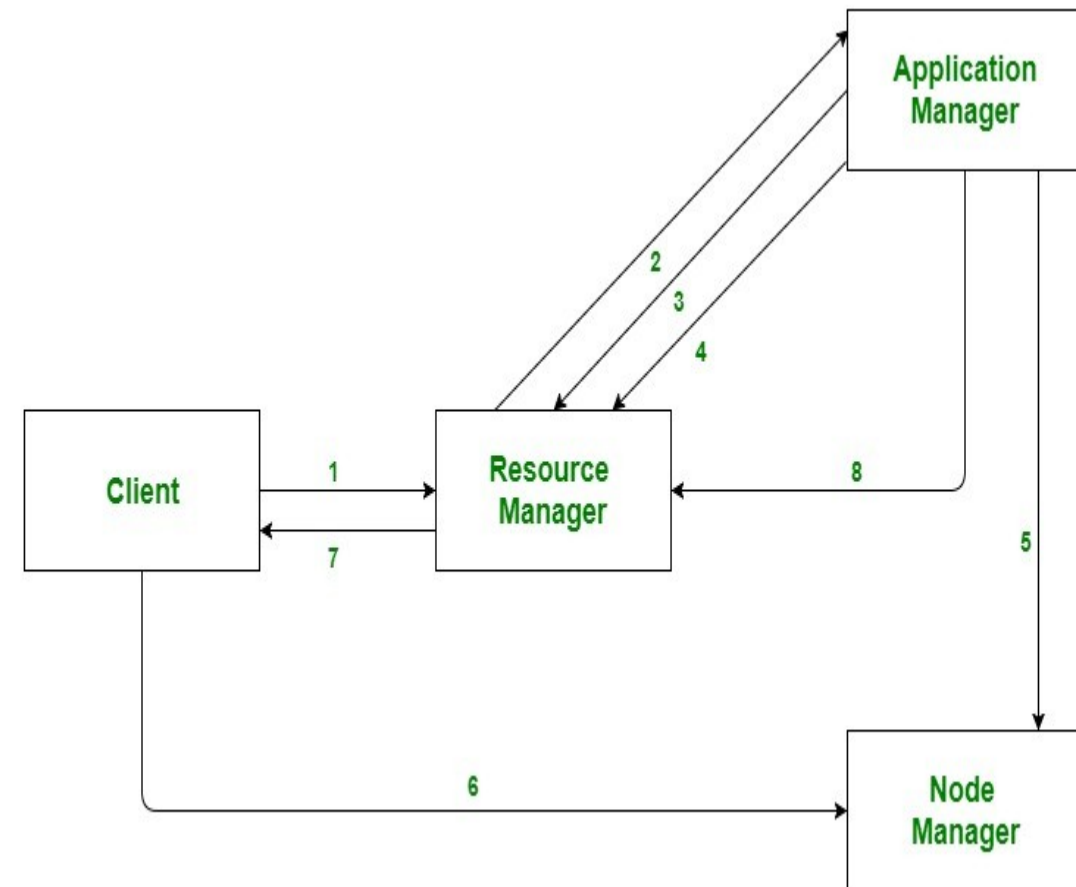


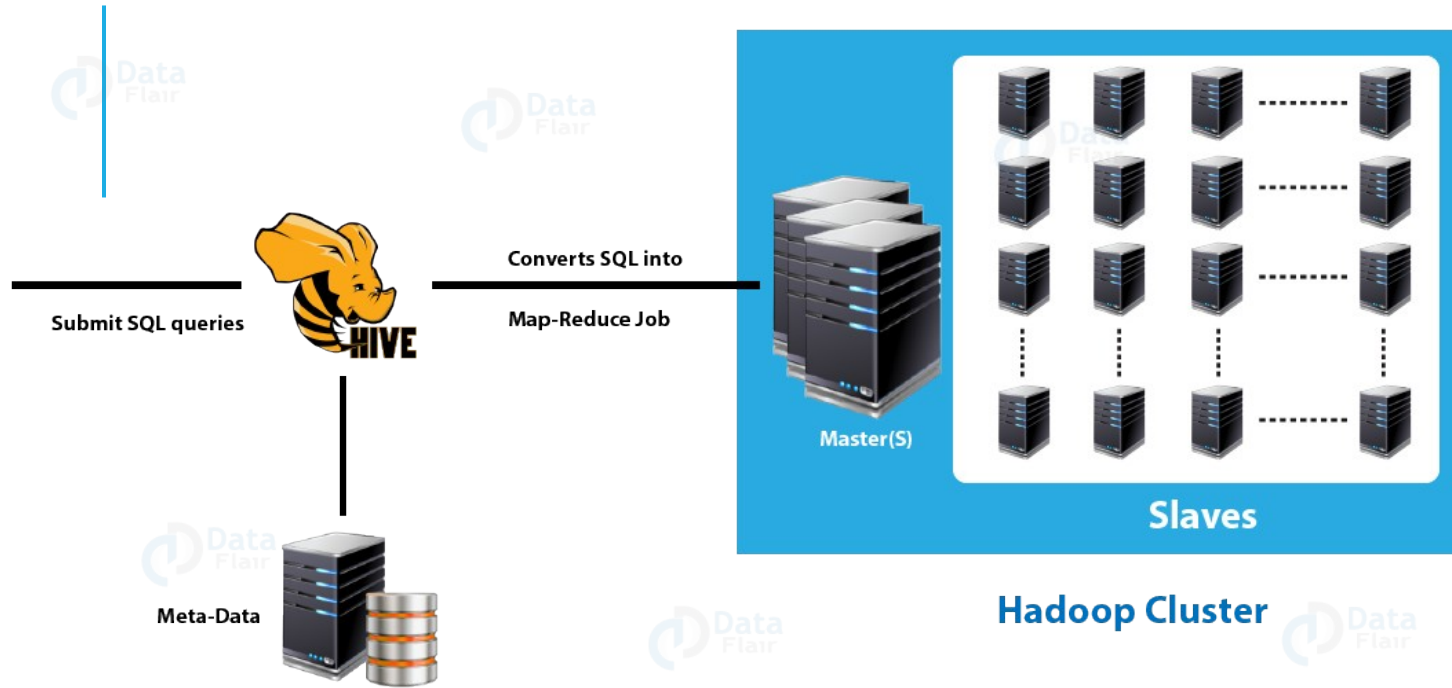
Application workflow in Hadoop YARN

Main Components:

- **Client:** Submits the job.
- **Resource Manager:** Decides where to run the job.
- **Application Manager:** Manages the submitted job.
- **Node Manager:** Runs the job on a worker node.

The client submits a job, the Resource Manager creates and launches an Application Manager, which requests containers from Node Managers to run the job, and updates are sent back to the client.





HIVE

Hive is a data warehouse project built on the top of Apache Hadoop **which provides data query and analysis.**

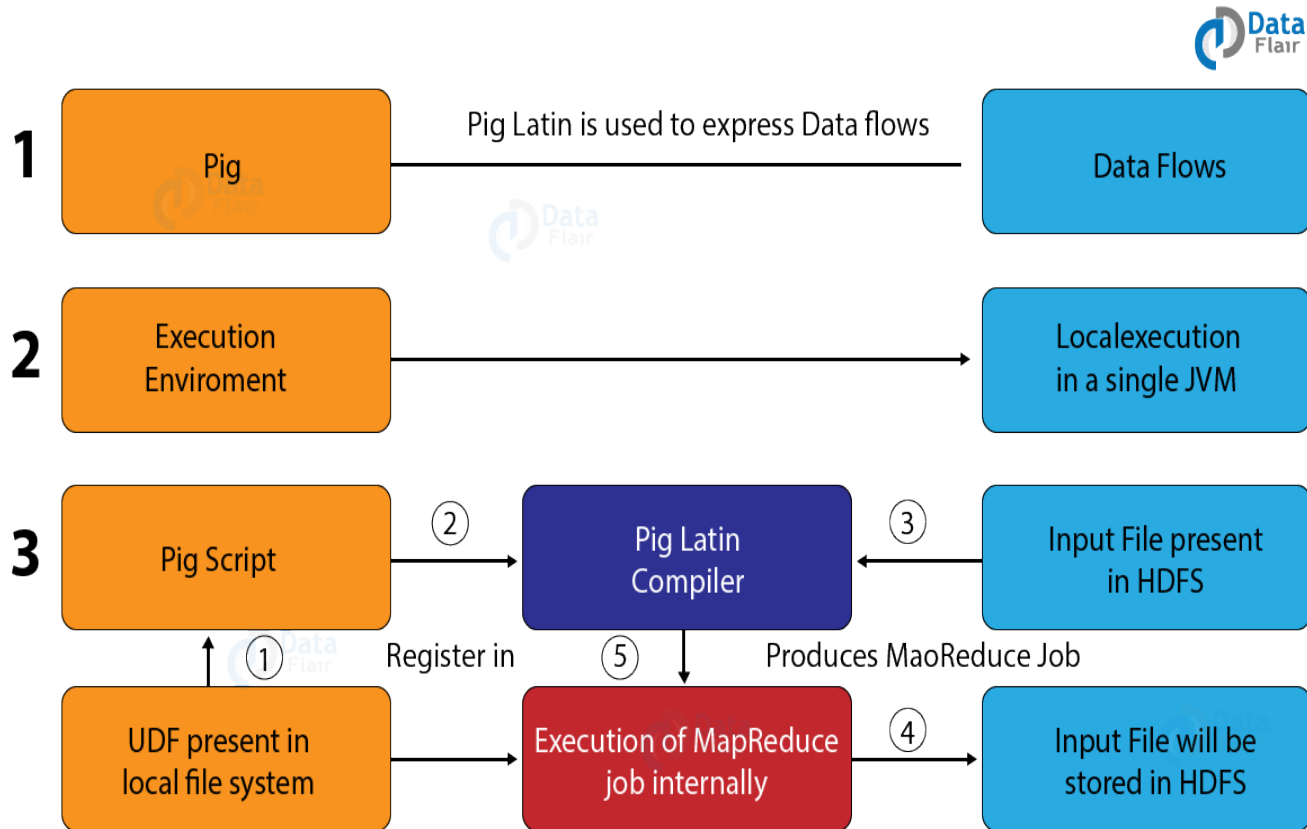
It has got the language of its own call HQL or Hive Query Language.

HQL automatically translates the queries into the corresponding map-reduce job.

Main parts of the Hive are -

- MetaStore – it stores metadata
- Driver – Manages the lifecycle of HQL statement
- Query compiler – Compiles HQL into DAG i.e., Directed Acyclic Graph
- Hive server – Provides interface for JDBC/ODBC server.
- Hive lets you **query big data in Hadoop using SQL** without writing complex Java MapReduce code. It translates your query into jobs and runs them on the Hadoop cluster.

PIG



Pig is a SQL like language used for querying and analyzing data stored in HDFS.

Yahoo was the original creator of the Pig.

It uses pig latin language.

It loads the data, applies a filter to it and dumps the data in the required format. Pig also consists of JVM called Pig Runtime.

Various features of Pig are as follows:-

Extensibility – For carrying out special purpose processing, users can create their own custom function.

Optimization opportunities – Pig automatically optimizes the query allowing users to focus on semantics rather than efficiency.

Handles all kinds of data – Pig analyzes both structured as well as unstructured.

Left Side (Orange Boxes): Inputs to the Pig Framework

Pig

The Pig framework itself, which provides an environment for executing Pig Latin scripts.

Execution Environment

This refers to the mode in which Pig runs:

Local mode: everything runs on a single machine using the local file system.

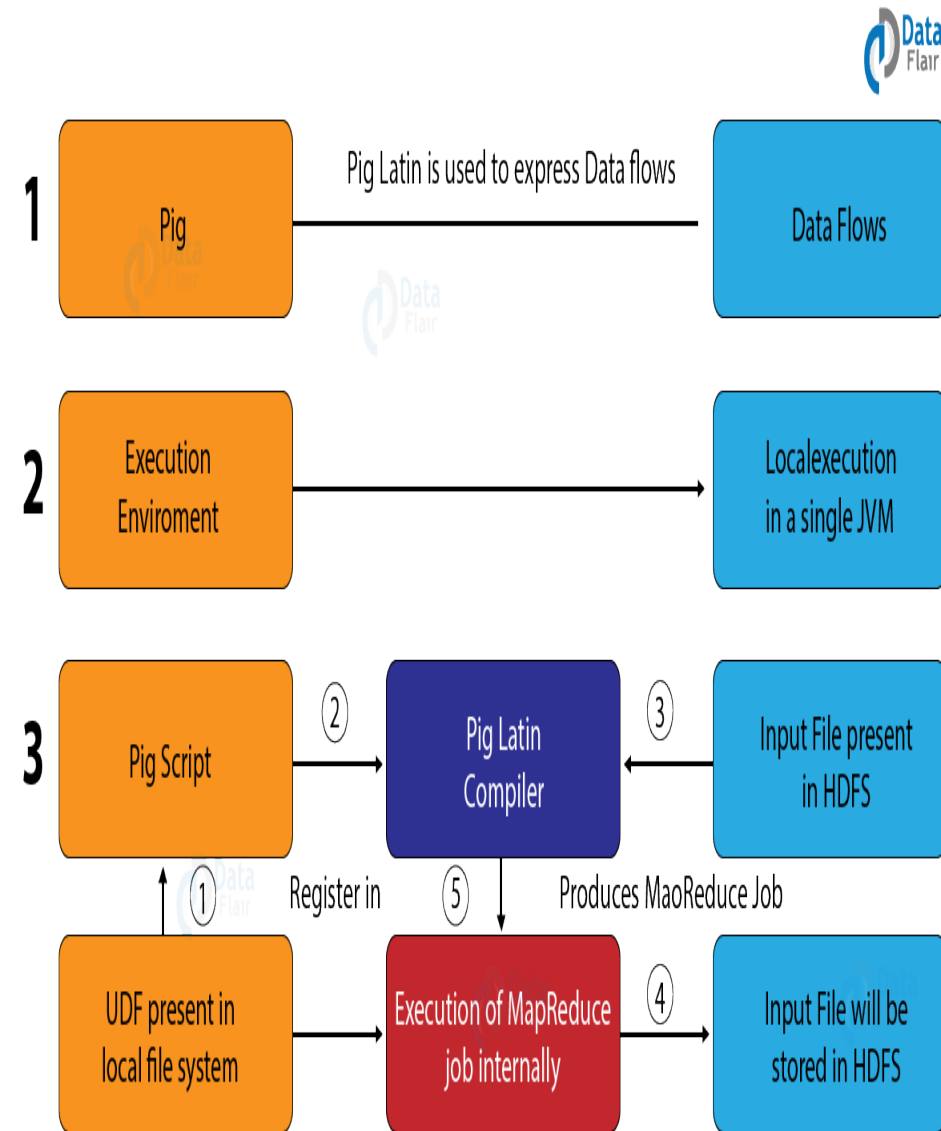
MapReduce mode: Pig converts scripts into MapReduce jobs and runs them on a Hadoop cluster.

Pig Script

A script written in **Pig Latin**, a data flow language similar to SQL but designed for parallel processing.

UDF present in local file system

User Defined Functions (UDFs) are custom functions written in Java (or other supported languages) to perform specific tasks not natively supported by Pig. These are loaded from the local file system.



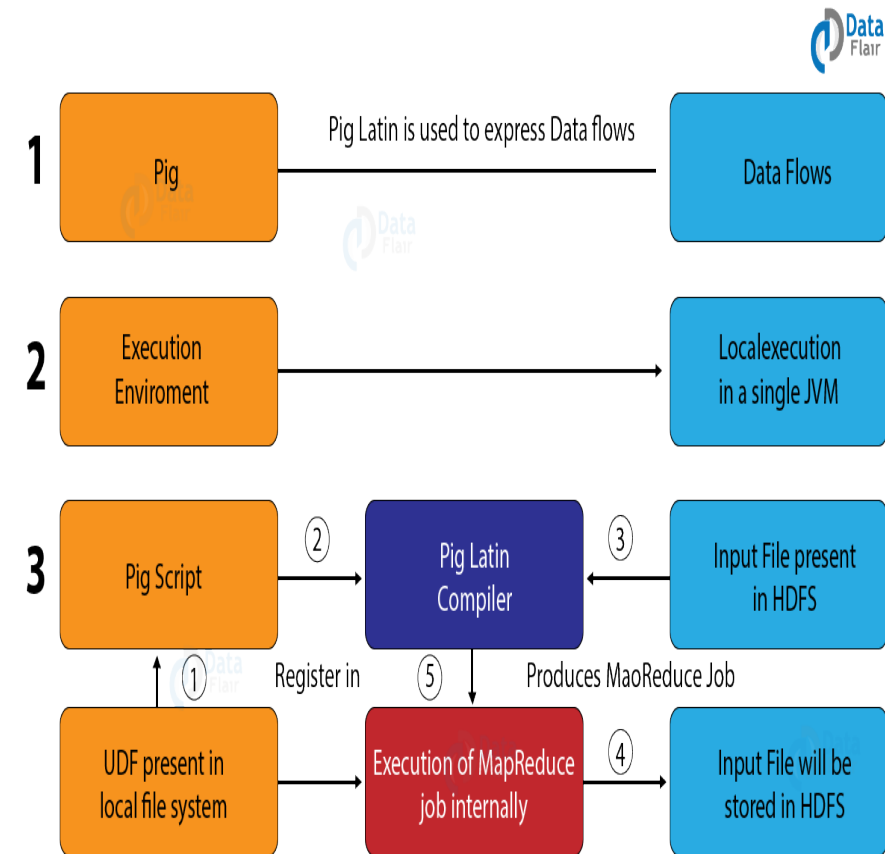
Middle Section (Execution Flow)

Pig Latin Compiler

Translates the Pig script (written in Pig Latin) into a sequence of MapReduce jobs.

Execution of MapReduce Job Internally

The compiled script is executed as MapReduce jobs within the Hadoop framework.



Right Side (Blue Boxes): Execution and Storage Details

Data Flows

Represents how data moves through the stages of the Pig script — from loading, transforming, and storing the data.

Local Execution in a Single JVM

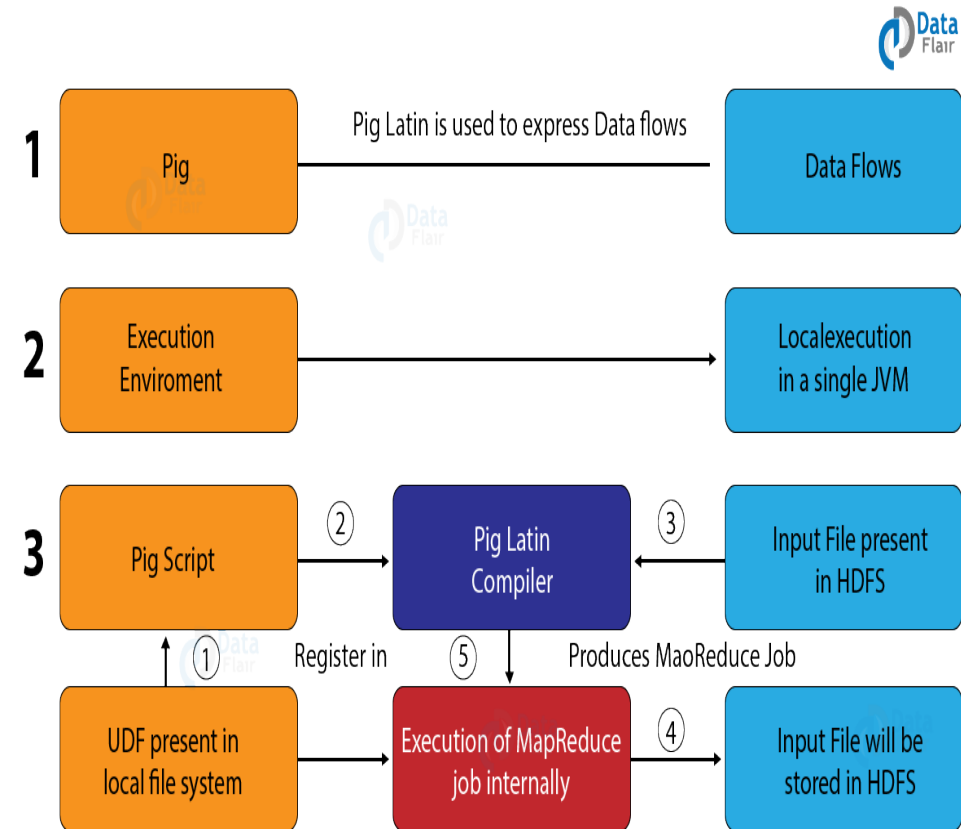
In **local mode**, Pig executes the script on a single machine within a single Java Virtual Machine (JVM), suitable for development or small data.

Input File Present in HDFS

In **MapReduce mode**, the input data resides in the **Hadoop Distributed File System (HDFS)**.

Input File will be Stored in HDFS

After processing, the output is stored back in HDFS for further use or querying.

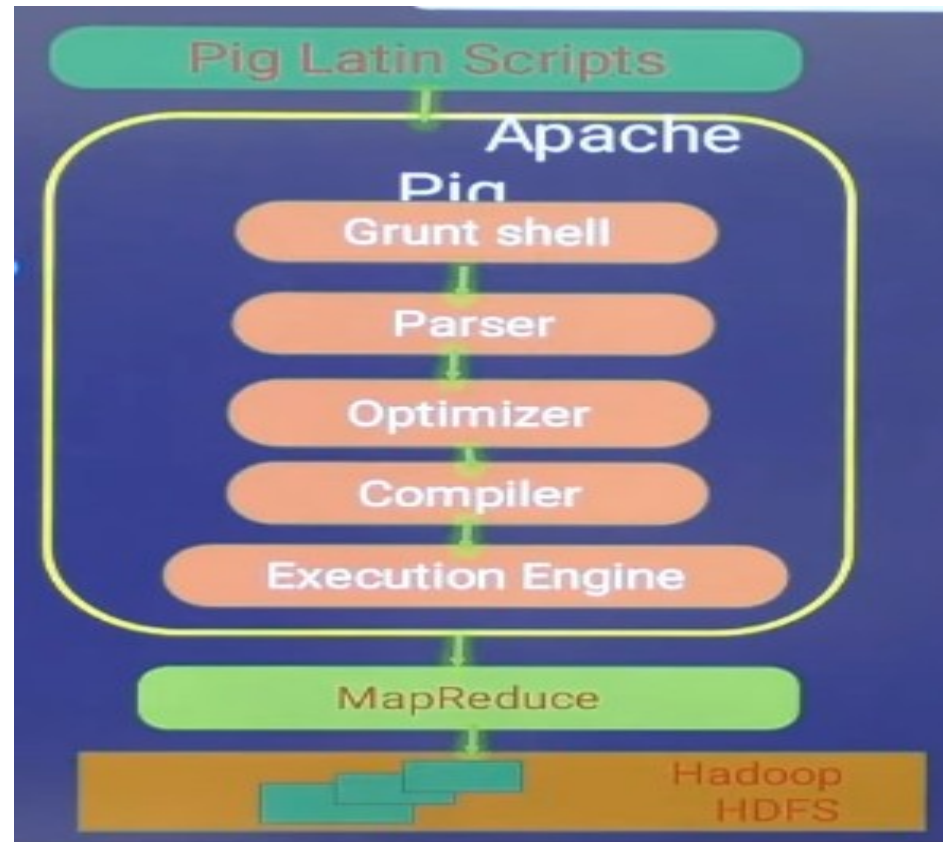


How does Pig work?

- First, the load command loads the data.
- At the backend, the compiler converts pig latin into the sequence of map-reduce jobs.
- Over this data, we perform various functions like joining, sorting, grouping, filtering etc.
- Now, you can dump the output on the screen or store it in an HDFS file.

Pig Architecture

<https://youtu.be/yDUcEmmJWxs?feature=shared>



Summary

Pig scripts written in Pig Latin are compiled into MapReduce jobs by the Pig compiler.

These scripts can use local or HDFS-based input files.

Execution can be in **local mode** (single JVM) or **MapReduce mode** (on a Hadoop cluster).

Pig supports UDFs for custom functionality.

Outputs are typically stored in HDFS.

HBASE



HBase is a NoSQL database built on the top of HDFS.

The various features of HBase are that it is open-source, non-relational, distributed database.

It imitates Google's Bigtable and written in Java. It provides real-time read/write access to large datasets. HBase is a column-oriented database management system. It runs on top of HDFS. It suits for sparse data sets which are common in Big Data use cases.

Its various components are as follows:-

HBase Master

HBase performs the following functions:

- Maintain and monitor the Hadoop cluster.
- Performs administration of the database.
- Controls the failover.
- HMaster handles DDL operation.

b. RegionServer

Region server is a process which handles read, writes, update and delete requests from clients. It runs on every node in a Hadoop cluster that is HDFS DataNode.



MAHOUT

Mahout provides a platform for creating machine learning applications which are scalable.

What Mahout does?

It performs collaborative filtering, clustering, and classification.

- Collaborative filtering – Mahout mines user behavior patterns and based on these it makes recommendations to users.
- Clustering – It groups together a similar type of data like the article, blogs, research paper, news etc.
- Classification – It means categorizing data into various sub-departments. For example, we can classify article into blogs, essays, research papers and so on.
- Frequent Itemset missing – It looks for the items generally bought together and based on that it gives a suggestion. For instance, usually, we buy a cell phone and its cover together. So, when you buy a cell phone it will give suggestion to buy cover also.

ZOOKEEPER

Zookeeper coordinates between various services in the Hadoop ecosystem. It saves the time required for synchronization, configuration maintenance, grouping, and naming.

Following are the features of Zookeeper:-

- Speed – Zookeeper is fast in workloads where reads to data are more than write. A typical read: write ratio is 10:1.
- Organized – Zookeeper maintains a record of all transactions.
- Simple – It maintains a single hierarchical namespace, similar to directories and files.
- Reliable – We can replicate Zookeeper over a set of hosts and they are aware of each other. There is no single point of failure. As long as major servers are available zookeeper is available.

Why do we need Zookeeper in Hadoop?

Hadoop faces many problems as it runs a distributed application. One of the problems is deadlock.

Deadlock occurs when two or more tasks fight for the same resource. Zookeeper solves Deadlock condition via synchronization.

Another problem is of race condition. This occurs when the machine tries to perform two or more operations at a time. Zookeeper solves this problem by property of serialization.



OOZIE



It is a workflow scheduler system for managing Hadoop jobs. It supports Hadoop jobs for Map-Reduce, Pig, Hive, and Sqoop. Oozie combines multiple jobs into a single unit of work. It is scalable and can manage thousands of workflows in a Hadoop cluster.

Oozie works by creating DAG i.e., Directed Acyclic Graph of the workflow. It is very much flexible as it can start, stop, suspend and rerun failed jobs.

Oozie is an open-source web-application written in Java. Oozie is scalable and can execute thousands of workflows containing dozens of Hadoop jobs.

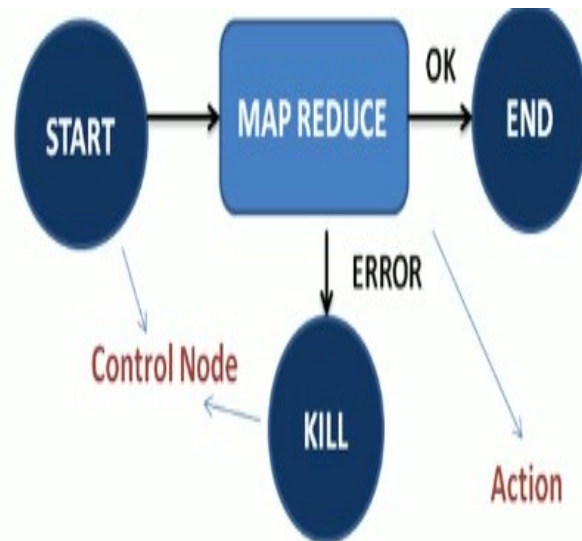
There are three basic types of Oozie jobs, and they are as follows:-

- Workflow – It stores and runs a workflow composed of Hadoop jobs. It stores the job as Directed Acyclic Graph to determine the sequence of actions that will get executed.
- Coordinator – It runs workflow jobs based on predefined schedules and availability of data.
- Bundle – This is nothing but a package of many coordinators and workflow jobs.

- Workflows consist of

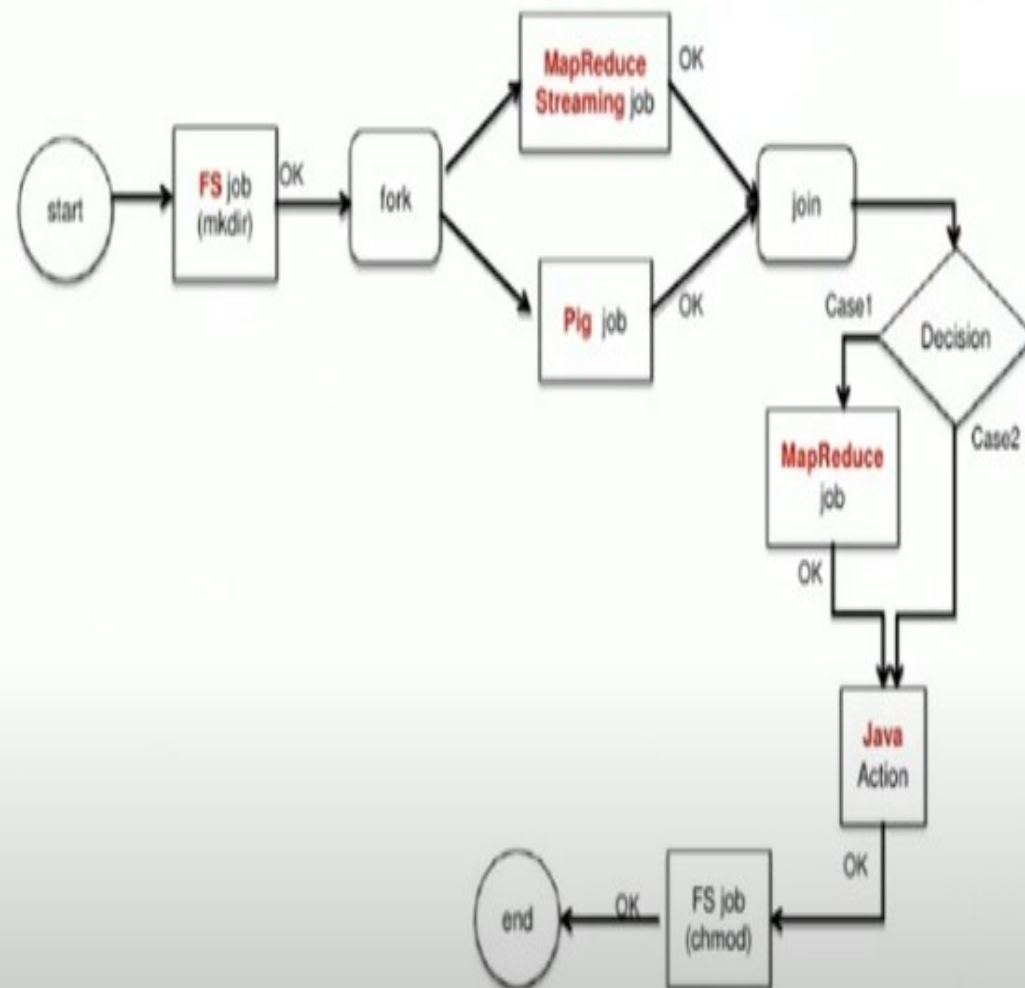
a) Action nodes

-MapReduce,Pig,Hive,Java etc.



b) Control Flow nodes

- Logic decisions between Action nodes.
- Execute actions based on conditions or parallel



SQOOP

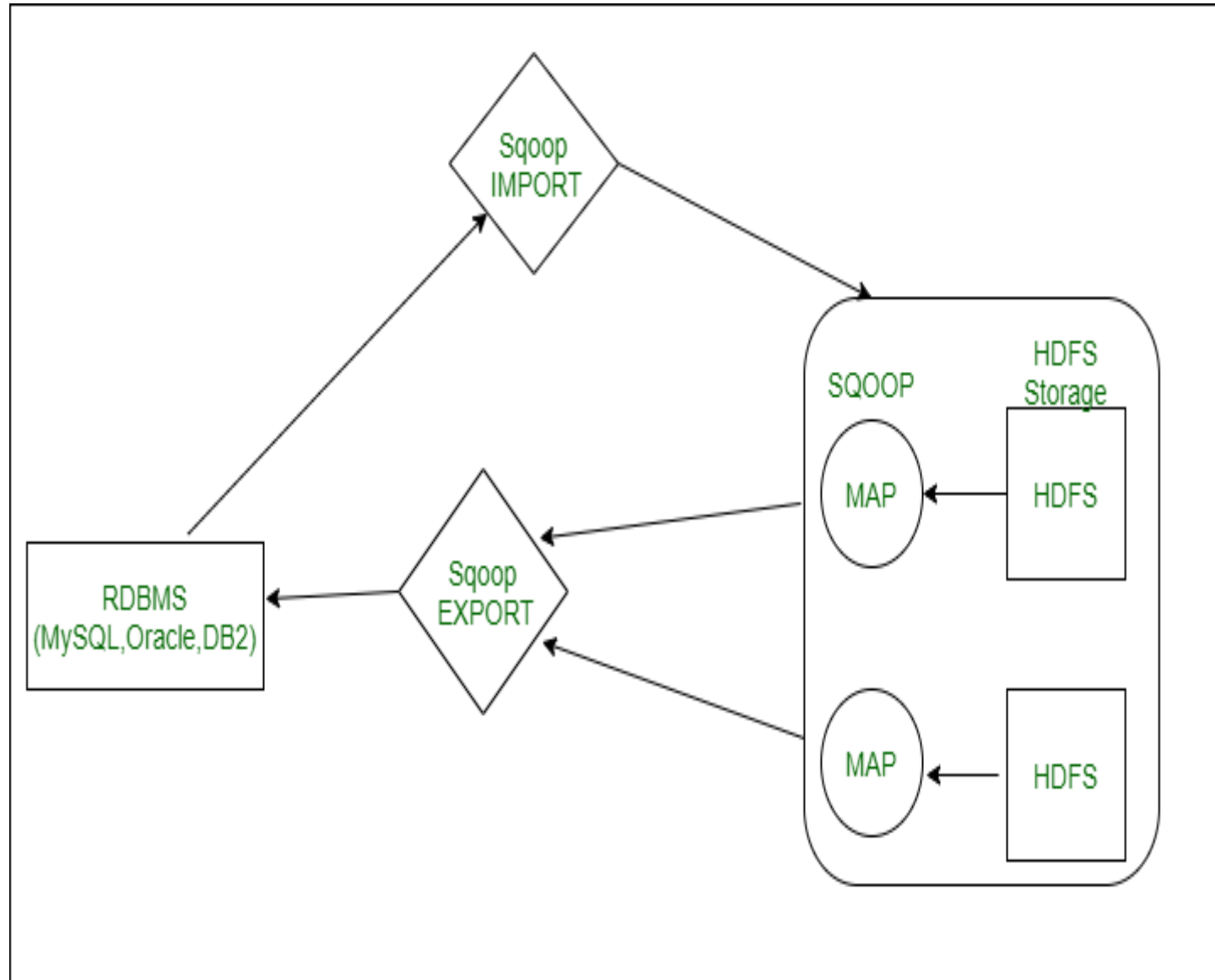


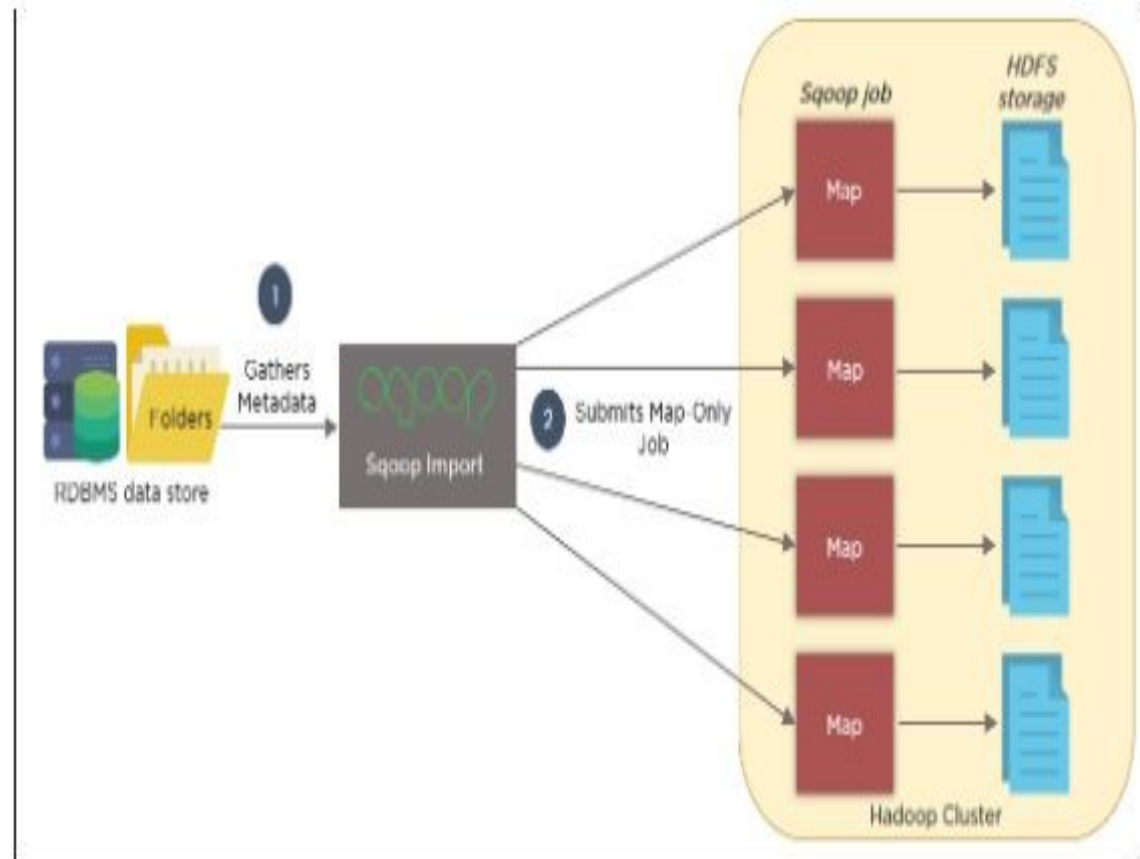
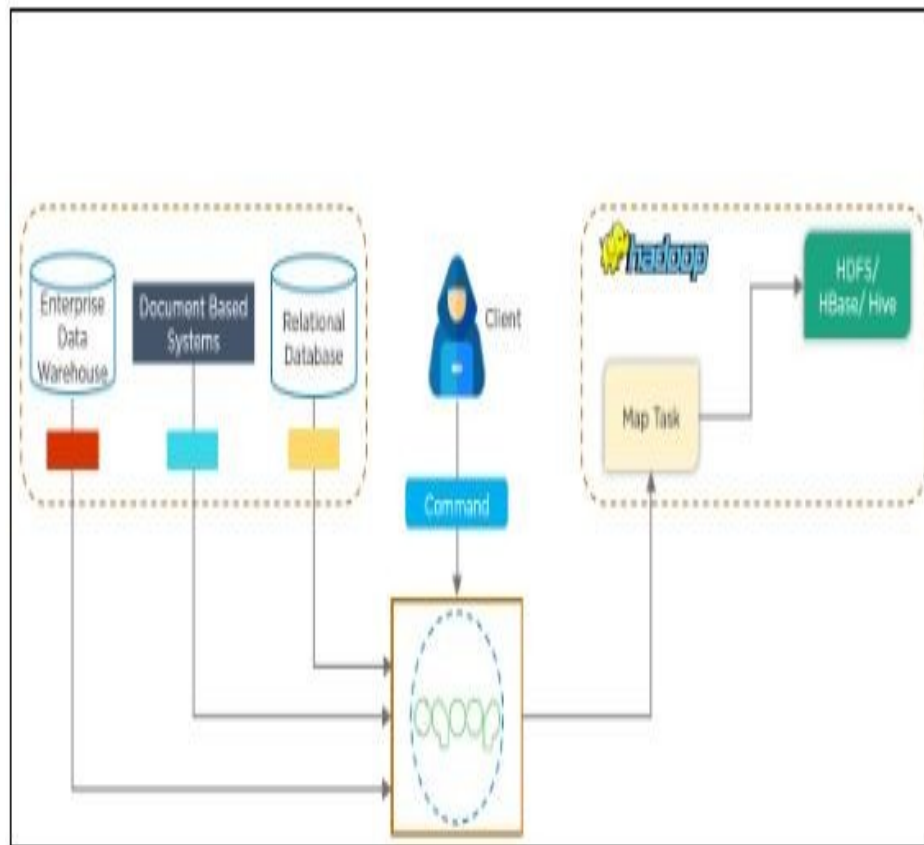
Sqoop imports data from external sources into compatible Hadoop Ecosystem components like HDFS, Hive, HBase etc.

It also transfers data from Hadoop to other external sources. It works with RDBMS like TeraData, Oracle, MySQL and so on.

The major difference between Sqoop and Flume is that Flume does not work with structured data.

<https://youtu.be/kSk8ILi5yT8?feature=shared>





import command on Hadoop

```
sqoop import \  
  --connect jdbc:mysql://hostname/database \  
  --username username \  
  --password password \  
  --table tablename \  
  --target-dir  
/user/hadoop/destination_directory \  
  --m 4
```

Suppose your MySQL table students has 1,000,000 rows.

--m 4

Sqoop may split it like:

- Mapper 1 → IDs 1 to 250000
- Mapper 2 → IDs 250001 to 500000
- Mapper 3 → IDs 500001 to 750000
- Mapper 4 → IDs 750001 to 1000000

Key Features and Components of Sqoop:

Connectivity: Sqoop provides connectors to various relational databases such as MySQL, Oracle, PostgreSQL, SQL Server, etc. These connectors allow Sqoop to establish connections to these databases and fetch data.

Import and Export: Sqoop supports both importing data from relational databases into Hadoop (into HDFS or Hive) and exporting data from Hadoop back into relational databases. This bidirectional capability makes it versatile for data integration tasks.

Parallelism: Sqoop can parallelize data transfer operations, making it efficient for handling large datasets. It automatically divides data into splits which are transferred in parallel, leveraging Hadoop's distributed computing capabilities.

Schema Inference: Sqoop can infer the schema of the tables it imports, which means it can automatically create the corresponding schema in Hadoop (e.g., Hive tables) based on the metadata retrieved from the relational database.

Incremental Imports: Sqoop supports incremental imports, where it can import only those rows from the database that have been added or updated since the last import. This helps in keeping the data in Hadoop up-to-date without transferring the entire dataset each time.

Customization: Sqoop provides options for customization such as specifying the number of mappers (parallel tasks), handling null values, specifying delimiters, etc., which allows users to tailor data transfer operations according to their needs.

FLUME

It is a service which helps to ingest structured and semi-structured data into HDFS. Flume works on the principle of distributed processing. It aids in collection, aggregation, and movement of a huge amount of data sets.

Flume has three components source, sink, and channel.

- Source – It accepts the data from the incoming stream and stores the data in the channel
- Channel – It is a medium of temporary storage between the source of the data and persistent storage of HDFS.
- Sink – This component collects the data from the channel and writes it permanently to the HDFS.



Ambari is another Hadoop ecosystem component. It is responsible for provisioning, managing, monitoring and securing Hadoop cluster. Following are the different features of Ambari:

Simplified cluster configuration, management, and installation

Ambari reduces the complexity of configuring and administration of Hadoop cluster security.

It ensures that the cluster is healthy and available for monitoring.

Ambari gives:-

Hadoop cluster provisioning

It gives step by step procedure for installing Hadoop services on the Hadoop cluster.

It also handles configuration of services across the Hadoop cluster.

Hadoop cluster management

It provides centralized service for starting, stopping and reconfiguring services on the network of machines.

Hadoop cluster monitoring

To monitor health and status Ambari provides us dashboard.

Ambari alert framework alerts the user when the node goes down or has low disk space etc.

APACHE SPARK

- Lightning fast cluster computing tool
- Apache Spark unifies all kinds of Big Data processing under one umbrella. It has built-in libraries for streaming, SQL, machine learning and graph processing. Apache Spark is lightning fast. It gives good performance for both batch and stream processing. It does this with the help of DAG scheduler, query optimizer, and physical execution engine.
- Spark offers 80 high-level operators which makes it easy to build parallel applications.
- Provides APIs in Scala, R, Java, Python

<https://youtu.be/OgS0noWVPJ4?feature=shared>

Before spark



Need for spark

- Need for a powerful engine that can process the data in Real-Time (streaming) as well as in Batch mode
- Need for a powerful engine that can respond in Sub-second and perform In-memory analytics
- Need for a powerful engine that can handle diverse workloads:
 - Batch
 - Streaming
 - Interactive
 - Graph
 - Machine Learning

Sort Record

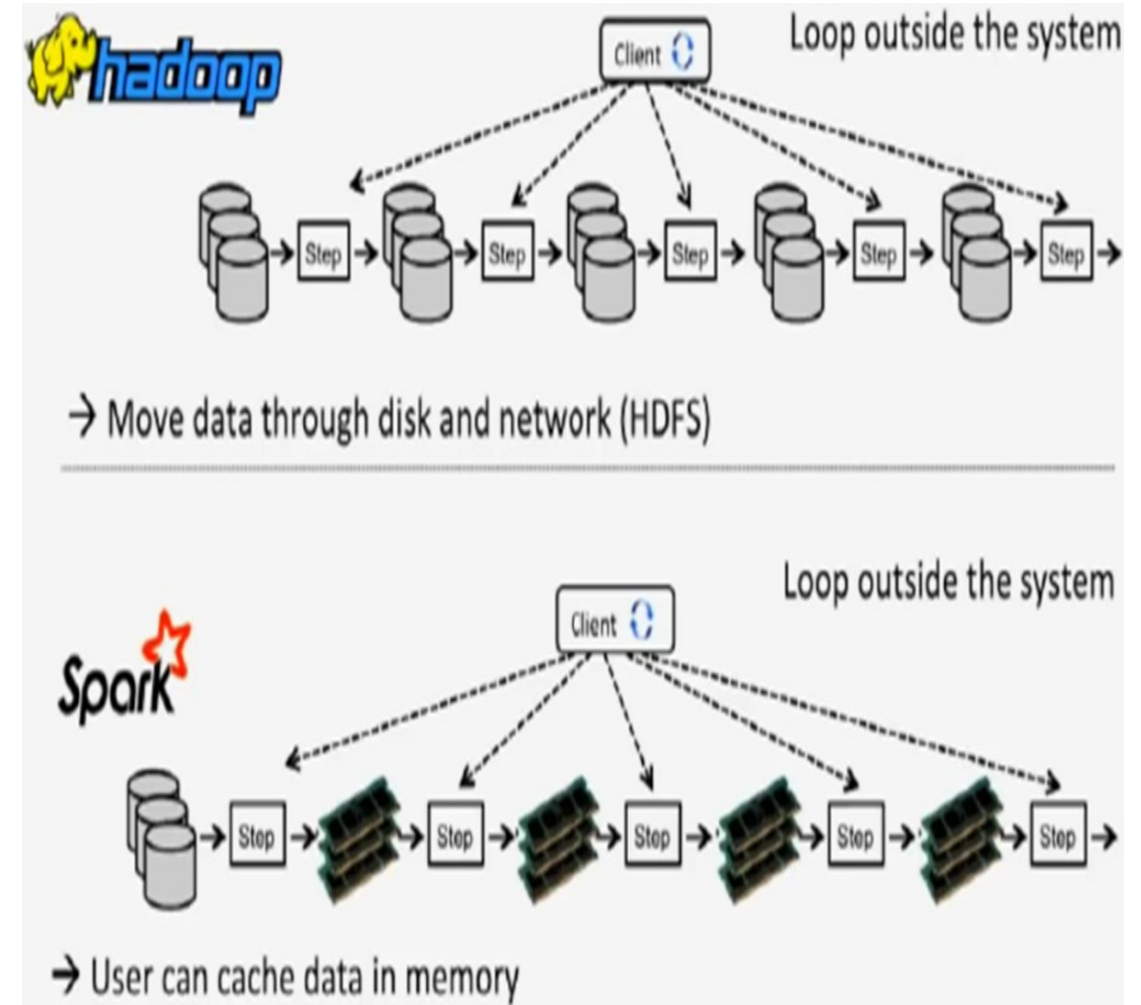


	Hadoop MapReduce	Spark
Data Size	102.5 TB	100 TB
Time Taken	72 min	23 min
No of nodes	2100	206
No of cores	50400 physical	6592 virtualized
Cluster disk throughput	3150 GBPS	618 GBPS
➡ Network	Dedicated 10 Gbps	Virtualized 10 Gbps

- Easy to develop sophisticated Algorithms
- Easy to combine batch, real time and graph processing
- Compatible with existing open source softwares

Why Spark

- 100x faster than Hadoop.
- In-memory computation.
- Language support like Scala, Java, Python and R.
- Support Real time and Batch Processing.
- Lazy Operations – optimize the job before execution.
- Support for multiple transformations and actions.
- Compatible with hadoop, can process existing hadoop data.





```
from pyspark import SparkConf, SparkContext
```

```
conf = SparkConf().setAppName("LazyEvaluationExample")
```

```
sc = SparkContext(conf=conf)
```

```
//loading data
```

```
# This line just sets up the logical plan, no data is read yet
```

```
text_file = sc.textFile("hdfs://path/to/textfile")
```




Define transformations

```
words = text_file.flatMap(lambda line: line.split(" ")) # Split lines into words
```

```
word_pairs = words.map(lambda word: (word, 1))          # Map each word to a (word, 1) pair
```

```
word_counts = word_pairs.reduceByKey(lambda a, b: a + b) # Reduce by key to count words
```

At this point, no computation has been executed yet.



```
# Perform an action to trigger execution  
output = word_counts.collect()
```

```
# Now the transformations are executed  
for (word, count) in output:  
    print(f"{word}: {count}")
```

Benefits of Lazy Evaluation

Reduced Number of Passes: Combining multiple transformations into a single stage reduces the number of passes over the data, saving time and computational resources.

Query Optimization: Allows Spark to analyze and optimize the logical plan before executing it, improving performance.

Improved Fault Tolerance: The logical plan provides lineage information, allowing Spark to recover from failures by recomputing only the affected parts.

GIST

At the core, we have HDFS for data storage, map-reduce for data processing and Yarn a resource manager. Then we have HIVE a data analysis tool, Pig – SQL like a scripting language, HBase – NoSQL database, Mahout – machine learning tool, Zookeeper – a synchronization tool, Oozie – workflow scheduler system, Sqoop – structured data importing and exporting utility, Flume – data transfer tool for unstructured and semi-structured data, Ambari – a tool for managing and securing Hadoop clusters

Summary of Hadoop Components

Tool	Purpose	Works With
Flume	Collect logs	Web logs → HDFS
Sqoop	Import/export RDBMS data	MySQL ↔ HDFS
Hive	Run SQL on big data	HDFS
Pig	Script-based processing	HDFS
Spark	Fast in-memory processing	HDFS
HBase	Real-time read/write database	HDFS
Zookeeper	Manage cluster coordination	HBase, Hive, Hadoop

REFERENCE

<https://data-flair.training/blogs/hadoop-ecosystem/>

[https://www.simplilearn.com/tutorials/hadoop-tutorial/hadoop-ecosyste
m](https://www.simplilearn.com/tutorials/hadoop-tutorial/hadoop-ecosystem)

What is NoSQL?

- Stands for "Not Only SQL".
- Non-relational database system for handling unstructured, semi-structured, or structured data.
- Designed for high scalability, flexibility, and distributed data architecture.
- Suitable for big data and real-time web applications.

NoSQL Data Architecture Patterns

1. Key-Value Stores

1. Store data as key-value pairs (e.g., Redis, Amazon DynamoDB).
2. Fast lookup using unique keys.
3. Ideal for caching, session management.

2. Graph Stores

1. Represent data as nodes and relationships (edges).
2. Useful for social networks, recommendation engines (e.g., Neo4j).

3. Column-Family Stores (Bigtable Model)

1. Store data in columns instead of rows.
2. High performance for read/write on large volumes (e.g., Apache HBase, Cassandra).

4. Document Stores

1. Store data as documents (e.g., JSON, BSON).
2. Flexible schema; suitable for hierarchical data (e.g., MongoDB, Couchbase).

MongoDB

- A popular document-based NoSQL database.
- Stores data in BSON format (binary JSON).
- Supports dynamic schemas and powerful querying.
- Excellent for content management, catalogs, IoT apps, etc.